队伍编号	MC2402980
题号	（D）

基于 QUBO 模型与量子计算的智慧矿山运营方案优化问题

摘要

随着智能技术的发展，矿山设备企业的运营及配置优化逐渐趋于智能化，其先进的技术基础可以实现高效地矿山开采。为提高矿山企业的核心竞争力，设计一套最优的设备配置及运营方案是目前具有重要应用价值的研究。

本文首先对基本的规划模型转化为 QUBO 模型进行了系统地总结归纳，并给出了相应的 QUBO 模型转化流程，为后续模型的转换提供了基础。

针对问题一，本文建立了以最大总利润为目标函数的优化模型，决策变量为不同挖掘机的采购数量和挖掘机与矿车的匹配关系 0-1 变量，约束条件取决于挖掘机和矿车各参数、启动资金及匹配机制的限制。基于该优化模型，将决策变量转化成对应的二进制变量表示形式，并根据约束条件加入相应的惩罚项，转化成最终的 QUBO 模型，并用 Kaiwu SDK 的模拟退火求解器和 CIM 模拟器进行求解，最终求得长期情况下的最大总利润为 **51000 万元**。

针对问题二，本题的最大总利润目标函数由挖矿收益与成本来决定，决策变量相较上题新增了不同挖掘机的矿车实际分配数，约束条件依然取决于挖掘机和矿车各参数、启动资金及匹配机制的限制。然后将其转化成 QUBO 模型，使用 Kaiwu SDK 的模拟退火求解器和 CIM 模拟器对 QUBO 模型进行求解，最终可得到相应的匹配关系和不同挖掘机的采购数量，模拟退火求解器和 CIM 模拟器解得的最大总利润分别为 **60193.73 万元**和 **60392.45 万元**。

针对问题三，在问题二模型的基础上，对挖掘机与矿车的各参数进行了维度扩充，因此本题优化模型的目标函数、决策变量及约束条件保持不变。同样将其转化成 QUBO 模型，但 QUBO 模型的变量数目较多，因此考虑采用**混合退火 subQUBO** 方法将该模型分解成多个子问题，利用 CIM 模拟器迭代各个子问题，并将子问题的解合并成最终的 QUBO 模型的解，转化可得相应的匹配关系和不同挖掘机的采购数量，最终得到最大总利润为 **52373.97 万元**。

针对问题四，通过查阅文献，拟提出如下场景——**基于 QUBO 模型的实时交通信号控制优化研究**。该应用场景具有实际意义和价值，复杂交通网的控制优化充分展示量子计算的优势，同时该场景有着对中国交通路网大规模优化应用的潜力。

最后，本文对模拟退火求解器和 CIM 模拟器的求解效率以及最优解的稳定性进行了**分析和评价**。本文以前三问 QUBO 模型的求解作为参考依据，针对两种求解器的平均求解时间与最优值波动状况进行可视化，结果显示 CIM 模拟器的求解效率明显优于模拟退火求解器；同时，CIM 模拟器的最优解波动很小，而模拟退火求解器波动幅度很大，由此可见 CIM 模拟器在量子计算上的优势很大。

关键词：QUBO 模型；量子计算；智慧矿山；混合退火 subQUBO 算法

目录

一、问题的背景与重述 1
1.1 问题背景 1
1.2 问题重述 1
二、问题分析 1
三、模型假设 3
四、符号及变量说明 3
五、模型准备 4
5.1 QUBO 模型 4
5.2 0-1 规划模型与 QUBO 模型转化 4
六、模型建立与求解 5
6.1 问题一模型的建立与求解 5
6.1.1 长期利润最大的优化模型建立 5
6.1.2 QUBO 模型的转化 6
6.1.3 QUBO 模型的求解 9
6.2 问题二模型的建立与求解 11
6.2.1 五年内利润最大的优化模型建立 11
6.2.2 QUBO 模型的转化 14
6.2.3 QUBO 模型的求解 16
6.3 问题三模型的建立与求解 17
6.3.1 五年内利润最大的优化模型重新建立 18
6.3.2 QUBO 模型的转化 19
6.3.3 QUBO 模型的求解 22
6.4 问题四的讨论——基于 QUBO 模型的实时交通信号优化控制研究 25
6.4.1 研究背景及意义 25
6.4.2 研究现状 26
6.4.3 创新点及研究思路 26
6.4.4 实时交通信号控制优化研究的 QUBO 模型 28
6.4.5 预期结果与应用 28
七、量子计算求解效率分析 29
7.1 较短的求解时间 29
7.2 较稳定的最优解 29
八、模型评价与改进 30
8.1 模型的优点 30
8.2 模型的缺点 30
8.3 模型的改进 30
九、模型推广与应用 31
参考文献 32
附录 33

一、问题背景与重述

1.1 问题背景

随着全球经济发展和资源需求的增长，矿山行业在满足人类对各种矿产资源的需求方面扮演着至关重要的角色。随着资源储量的下降、开采成本的增加以及环境保护要求的提高，传统的矿山运营模式面临着严峻的考验，如何合理配置矿山设备，并优化运营方案成为了一个亟待解决的问题。传统的设备配置和运营方案往往依赖于经验和启发式方法，无法在处理复杂的优化问题时提供最优解。近年来，智慧矿山的概念越受到重视，建立“协同+生态”的智慧矿山系统能有效提高企业的核心竞争力。智慧矿山依靠先进的技术和设备自动化，可以实现高效、安全、环保的矿山开采。量子计算作为一种新兴的计算范式，为解决上述复杂的优化问题提供了新的可能性。量子计算以其能够并行处理和量子叠加的特性，能够在理论上提供比传统计算方法更高效的解决方案。

本研究旨在利用量子计算技术，建立 QUBO（Quadratic Unconstrained Binary Optimization，二次无约束二值优化）模型，转化传统优化问题为 QUBO 形式，应用玻色量子公司提供的 Kaiwu SDK 套件。通过对矿山设备的合理配置和运营方案的优化，旨在提高智慧矿山的效率和可持续性，为解决全球资源供需不平衡问题做出贡献。

1.2 问题重述

假定我们是智慧矿山项目团队，为一家即将投入运营的智慧矿山设计一个综合的设备配置和运营方案。需要考虑挖掘机斗容、挖掘机作业效率、矿车装载量、油耗、价格、人工成本、维护成本以及项目规模和设备数量因素，最终得到采最优方案。

问题一的重述：不考虑挖掘机的使用寿命的前提下，利用挖掘机的长期利润折现值估计，建立简化场景的 QUBO 模型，并通过 Kaiwu SDK 内置的模拟退火求解器以及 CIM 模拟器求解预算范围内的总利润最大时的采购方案。

问题二的重述：挖掘机和矿车的使用寿命为 5 年前提下，考虑题中约束，建立 QUBO 模型，通过模拟退火求解器和 CIM 模拟器进行求解，给出最优采购的挖掘机型号和数量以及挖掘机与矿车的匹配关系，对比特数超出限制时，给出创新的求解方案。

问题三的重述：当矿车、挖掘机种类，挖掘机型号及匹配关系更为复杂的情况下，建立 QUBO 模型，并使用 Kaiwu SDK 对模型进行求解，给出最优采购的挖掘机型号和数量以及挖掘机与矿车的匹配关系，可以考虑采用 subQUBO 方法进行求解。

问题四的重述：例举一个可以用 QUBO 模型进行决策优化的场景，要求场景具有实际意义，展示量子计算的优势，有规模化应用的潜力。（需要给出项目的背景信息、研究方法、研究思路、预期结果、技术路图、QUBO 模型表达式等）

二、问题分析

2.1 问题一的分析

在不考虑挖掘机的使用寿命的情况下，通过考虑每种类型挖掘机长期利润的折现值，决策变量为采购挖掘机的型号及其对应数量，目标函数为预算范围内总利润最大化。根据题目中给出的有关约束条件，构建优化模型。然后将该模型转化为 QUBO 模型。最后利用 Kaiwu SDK 内置的求解器进行求解，分析比较求解的结果。

2.2 问题二的分析

为解决挖掘机和矿车再固定 5 年寿命的前提下下的总利润最大化的问题，要求给出挖掘机和矿车之间的匹配关系以及采购的挖掘机型号及其对应数量，需要对第一问的简单模型进行拓展。决策变量为采购挖掘机的型号及其对应数量，目标函数为总利润的最大化。考虑题目中的有关约束条件），构建优化模型。然后将该模型转化为 QUBO 模型。最后利用 Kaiwu SDK 的求解器进行求解，分析比较求解结果（最优采购的挖掘机型号数量及挖掘机与矿车的匹配关系），对比特数超出限制时，给出创新的求解方案。

2.3 问题三的分析

为解决挖掘机和矿车固定 5 年寿命的情况下的总利润最大化的问题，要求给出挖掘机和矿车之间的匹配关系以及采购的挖掘机型号及其对应数量。决策变量为采购挖掘机的型号及其对应数量，目标函数为总利润最大化，构建优化模型，然后将该模型转化为 QUBO 模型。利用 Kaiwu SDK 内置的求解器进行求解。分析求解结果，同时考虑采用 subQUBO 等方法进行求解。

问题三实际上是问题二的推广形式，增加了可选挖掘机的数量、可选矿车的数量及其匹配关系等条件，同时要求对挖掘机的种类不少于 5 种，且启动资金为 4000 万元。要修改部分的约束条件，以建立原始的优化模型。

2.4 问题四的分析

题目中要求我们例举一个可以用 QUBO 模型进行决策优化的场景，通过查询文献，我们拟提出如下场景——基于 QUBO 模型的实时交通信号控制优化研究。该场景具有实际意义和价值，复杂交通网的控制优化可以展示量子计算的优势，同时该场景有着对中国的交通路网优化规模化应用的潜力。

我们得到问题分析流程图如下：

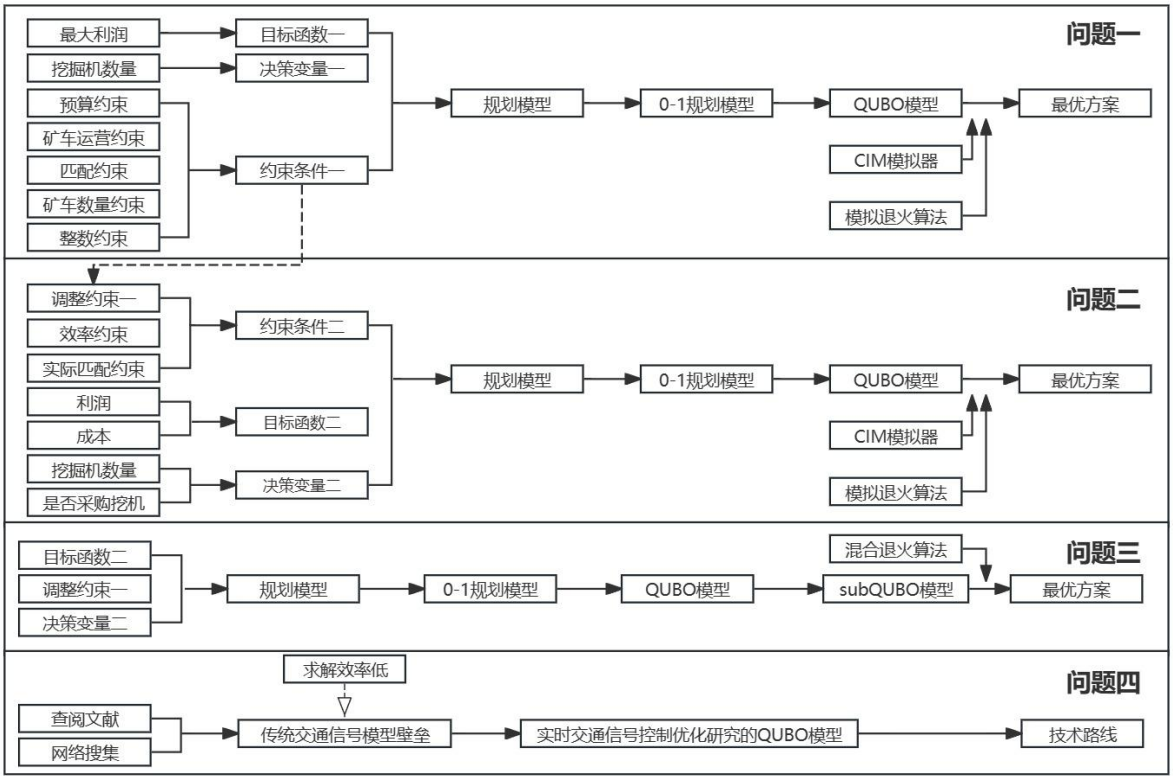

图 1 问题分析流程图

三、模型假设

1. 假设在不考虑挖掘机使用寿命的问题中，每台挖掘机的长期利润折现值指的是在相当长的一段时间内，挖掘机可以带来的总利润，即不考虑其余的人工成本和维护成本。

2. 假设在不考虑挖掘机使用寿命的问题中，为保证长期使用效益最大化，挖掘机需要与给定数量的矿车始终保持匹配关系，且匹配关系不变。

3. 假设挖掘机和矿车的效率、挖掘机和矿车的匹配关系都保持不变。同型号的挖掘机只能匹配统一型号的矿车

4. 假设采购挖掘机仅在第一年，后续无需考虑挖掘机的购买费用。

四、符号及变量说明

符号及变量	说明	单位
$n_i^{(1)}$	第 i 型挖掘机的采购量	台
$x_i^{(1)}$	第 i 型挖掘机是否采购	/
y_{ij}	第 j 型矿车是否和第 i 型挖掘机匹配	/
Z_l	长期总利润	万元
Z	总利润	万元
ers	总收益	万元
op_k	成本的子模块 k	万元
$C_i^{(1)}$	第 i 型挖掘机的采购费用	万元
E_i	第 i 型挖掘机的作业效率	万元
V_i	第 i 型挖掘机的斗容	万元
$M_i^{(1)}$	第 i 型挖掘机的维护成本	万元
$P_i^{(1)}$	第 i 型挖掘机的人工成本	万元
$O_i^{(1)}$	第 i 型挖掘机的油耗费用	万元
$n_j^{(2)}$	矿车实际采购量	台
$M_j^{(2)}$	第 j 型矿车的维护成本	万元
$P_j^{(2)}$	第 j 型矿车的人工成本	万元
$O_j^{(2)}$	第 j 型矿车的维护成本	万元
M_5	五年的挖矿总量	立方米
P	惩罚项系数	/
Q	QUBO 矩阵	/
N_j	矿车数量	台
$b_k^{[i]}$	$n_i^{(1)}$ 的二进制表示	/
R_{ij}^*	第 i 型挖掘机理想情况下匹配到第 j 型矿车的数目	台
R_{ij}	第 i 型挖掘机实际情况下匹配到第 j 型矿车的数目	台

五、模型准备

5.1 QUBO 模型

QUBO 模型[1]是一种二次无约束模型，用于解决组合优化问题。在 QUBO 模型中，问题被表述为一个关于二进制变量的二次函数的优化问题。每个二进制变量可以取值为 0 或 1，代表某种决策的选择。

QUBO 模型的一般形式：$QUBO: \max Z = xQx^T \quad s.t. x \ \ binary$

其中，x 是一个包含二进制变量的向量，Q 是一个二次型的矩阵形式，它描述了问题的目标函数中各个变量之间的关系和影响。

5.2 0-1 规划模型与 QUBO 模型转化

由于 QUBO 模型是二次无约束模型，对于一个 0-1 规划模型，只需将其目标函数一次项系数按照特定约束转化为二次项，并将约束条件以惩罚项的形式加入目标函数即可转化为 QUBO 模型[1]，具体转化规则如下：

(1)目标函数一次项的转化

由于变量的取值只能为 0,1，为此对其平方并不影响结果，即 $x \in \{0,1\}, x^2 = x$ 。

(2)约束条件的转化

对于约束条件的转化，分为简单约束和多元约束进行考虑。

对于简单约束，对应的约束规则如下表，P 为惩罚系数：

表 1 简单约束转化为目标函数惩罚项表

简单约束	转化后的惩罚项
$x+y \le 1$	$P(xy)$
$x+y \ge 1$	$P(1-x-y+xy)$
$x+y=1$	$P(1-x-y+2xy)$
$x=y$	$P(x+y-2xy)$

对于多元约束，如若为等式约束，对等式进行移项使得右端项为 0，使其平方后意义不发生改变，以此得到惩罚项 $P\sum_i(\sum_{j=1}^n a_{ij}x_{ij}-b_i)^2$.如若为不等式约束，我们需要将其加入松弛变量转化为等式变量，得到新的 0-1 等式约束以此继续进行转化，流程如下：

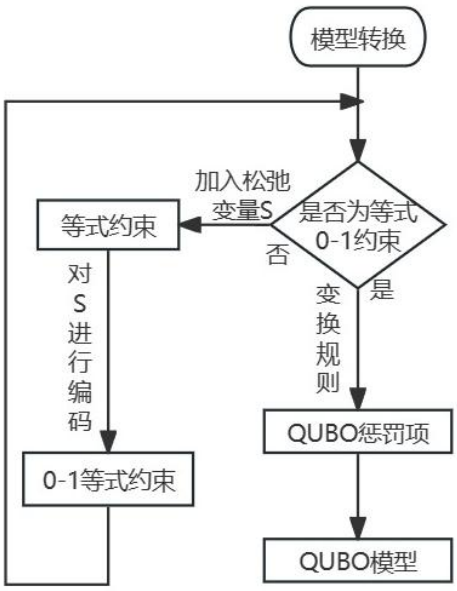

图 2 0-1 规划模型与 QUBO 模型转化图

六、模型的建立与求解

6.1 问题一模型的建立与求解

在不考虑挖掘机的使用寿命的情况下，考虑每种类型挖掘机长期利润的折现值，决策变量为采购挖掘机的型号及其对应数量。目标函数为预算范围内总利润最大化。考虑题目中给出的有关约束条件，构建优化模型。然后将该模型转化为 QUBO 模型。最后利用 Kaiwu SDK 内置的模拟退火求解器和 CIM 求解器进行求解。

6.1.1 长期利润最大的优化模型建立

为构建长期利润最大的优化模型，需要确定模型的决策变量、目标函数及约束条件。

(1) 决策变量

决策变量 1, $n_i^{(1)}(i=1,2,3,4)$：第 i 型挖掘机的采购量（单位：台）。

决策变量 2, $x_i^{(1)} \in \{0,1\}$：第 i 型挖掘机是否采购，其中 $x_i^{(1)}=0$ 代表不采购，$x_i^{(1)}=1$ 代表采购（单位：/）。

(2) 目标函数

四种不同型号产生的总长期利润进行折现估计，写出长期的总利润 Z_l 的表达式。

求长期利润最大，目标函数如下：

$$\max Z_l = \sum_{i=1}^{4} n_i^{(1)} x_i^{(1)} A_i^{(1)} \tag{1}$$

其中 $A_i^{(1)} = (2000, 3000, 5000, 6000)^T$ 表示第 i 型挖掘机长期利润折现值（单位：万元）。

(3)约束条件

由于考虑的是长期利润折现的情况，并不知道计划开采时间，所以不考虑人工成本、维护成本、油耗等成本约束；仅考虑启动资金预算、矿车运营匹配、矿车数量及效率、挖掘机购买数量的整数约束。

约束条件 1：启动资金预算约束。在制定长期采购方案时，由于项目启动资金有限，在采购挖掘机时要考虑资金，约束如下：

$$\sum_{i=1}^{4} C_i^{(1)} n_i^{(1)} \le 2400 \tag{2}$$

其中，$C_i^{(1)}$ 为第 i 型挖掘机的采购价格（单位：万元）。

约束条件 2：矿车运营约束。出于实际运营的考量，整体包含的挖掘机型号不能少于三种，约束如下：

$$\sum_{i=1}^{4} x_i^{(1)} \ge 3, x_i^{(1)} \in \{0,1\} \tag{3}$$

约束条件 3：挖掘机和矿车的匹配约束。在挖掘机作业中，考虑到挖掘机铲斗宽度和矿车宽度的对应关系以及装车效率，需要满足特定的挖掘机与矿车的匹配关系（题中已给出）。

基于给出的矿车及挖掘机型号匹配绑定假设，即每个型号只对应一种矿车，对此我们引入一个 0-1 变量 $y_{ij}(i=1,2,3,4; j=1,2,3)$。$y_{ij}$ 表示第 j 型矿车是否和第 i 型挖掘机匹配。1 为匹配。0 为不匹配。结合题中数据给出以下约束：

$$\begin{cases} \sum_{j=1}^{3} y_{ij} = 1, \forall i = 1,2,3,4, y_{ij} \in \{0,1\} \\ y_{12} = y_{13} = y_{23} = y_{41} = 0 \end{cases} \tag{4}$$

其中，由于同一类挖掘机可以和不同类型的矿车进行匹配，但是匹配关系固定不变，

而且同一型号挖掘机只能匹配同一信号的矿车，所以可以得到$\sum_{j=1}^{3} y_{ij} = 1, \forall i = 1,2,3,4$。且不能匹配的挖掘机和矿车的$y_{ij}$必须为 0.

约束条件 4：挖掘机与矿车匹配后的矿车数量约束。我们需要考虑挖掘机和矿车匹配后矿车数量不能大于已有的矿车数量。

考虑引入一个 4 行 3 列的矩阵$R_{4\times3}^{*}$：

$$R^{*} = \begin{pmatrix} 1 & \infty & \infty \\ 2 & 1 & \infty \\ 2 & 1 & 1 \\ \infty & 2 & 1 \end{pmatrix} \tag{5}$$

其中，R_{ij}^{*}表示第i型挖掘机理想情况下匹配到第j型矿车的数目，即效率为 100%的情况，R_{ij}表示第j型矿车和第i型挖掘机匹配个数。由于本模型为长周期的最大利润模型，既要保证挖掘机的效率又防止空闲矿车造成资源浪费，故此$R_{ij} = R_{ij}^{*}$。∞表示挖掘机与矿车不存在匹配关系，$N_j = (7,7,3)^T$表示矿车数量，y_{ij}表示第i型挖掘机是否与第j型矿车匹配（若匹配，则$y_{ij} = 1$，否则$y_{ij} = 0$），约束如下：

$$\sum_{i=1}^{4} n_i^{(1)} y_{ij} R_{ij}^{*} \le N_j, \forall j = 1,2,3 \tag{6}$$

约束条件 5：挖掘机购买数量的整数约束。由于挖掘机的购买数量为整数，约束如下：

$$n_i^{(1)} \in \mathbb{Z}^{+} (i = 1,2,3,4) \tag{7}$$

综上所述，可以构建期总长期利润最大的优化模型如下，见式(8):

$$\begin{aligned} &\max Z_l = \sum_{i=1}^{4} n_i^{(1)} x_i^{(1)} A_i^{(1)} \\ &s.t. \begin{cases} x_i^{(1)} \in \{0,1\}, y_{ij} \in \{0,1\} \\ \sum_{i=1}^{4} x_i^{(1)} \ge 3 \\ \sum_{i=1}^{4} C_i^{(1)} n_i^{(1)} \le 2400, n_i^{(1)} \in \mathbb{Z}^{+} (i = 1,2,3,4) \\ \sum_{i=1}^{4} n_i^{(1)} y_{ij} R_{ij}^{*} \le N_j, \forall j = 1,2,3, n_i^{(1)} \in \mathbb{Z}^{+} (i = 1,2,3,4) \\ \sum_{j=1}^{3} y_{ij} = 1, \forall i = 1,2,3,4 \\ y_{12} = y_{13} = y_{23} = y_{33} = y_{41} = 0 \end{cases} \end{aligned} \tag{8}$$

6.1.2 QUBO 模型的转化

在 6.1.1 中，已经建立了总长期利润最大的优化模型，现在需要将该优化模型转化为 QUBO 模型的形式，便于使用 Kaiwu SDK 中的内置求解器进行求解。在 6.1.2 中，考虑分步对原始的总长期利润最大的优化模型转化为 QUBO 模型。

给出 QUBO 模型的转化流程，见图 2：

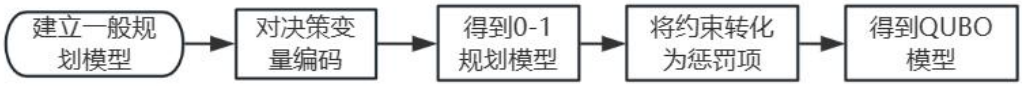

图 3 QUBO 模型转化流程

(1) 转化上述原始模型为 0-1 规划模型

Step1：将 $n_i^{(1)}x_i^{(1)}$ 合并为单一的变量 $n_i^{(1)}$

第 i 型挖掘机的采购量 $n_i^{(1)}$ 和第 i 型挖掘机是否采购 $x_i^{(1)}$ 可以用 $n_i^{(1)}$ 进行统一，即 $n_i^{(1)}=0$ 时表示第 i 型挖掘机不采购。统一之后可以去掉变量 $x_i^{(1)}$， $n_i^{(1)}\in\mathbb{Z}^+\cap 0$ 。

Step2：对 $n_i^{(1)}$ 的取值进行估测并对其进行二进制转化

根据约束条件 $\sum_{i=1}^{4}C_i^{(1)}n_i^{(1)}\le 2400$ ，可以估测出 $n_i^{(1)}\in\{0,1,2,...,14,15\}$ 。

任意 $n_i^{(1)}$ ，可用四个 0-1 变量表示，例: $n_1^{(1)}=b_1^{[1]}+2b_2^{[1]}+4b_3^{[1]}+8b_4^{[1]},b_k^{[1]}\ binary$ 。可知如下二进制转换公式：

$$n_i^{(1)}=\sum_{k=1}^{4}2^{k-1}b_k^{[i]}\in\{0,1,2,...,14,15\},b_k^{[i]}\in\{0,1\} \tag{9}$$

经过转换后，将原优化问题的决策变量转化为 $b_k^{[i]}\ binary$ ，这是一个一般的 0-1 规划问题，写出该 0-1 规划问题如下：

$$\begin{aligned}&\max Z_l=\sum_{i=1}^{4}\sum_{k=1}^{4}2^{k-1}b_k^{[i]}A_i^{(1)}\\&s.t.\begin{cases}y_{ij}\in\{0,1\},b_k^{[i]}\in\{0,1\}\\\sum_{j=1}^{3}y_{ij}=1,\forall i=1,2,3,4\\y_{12}=y_{13}=y_{23}=y_{41}=0\\\sum_{i=1}^{4}\sum_{k=1}^{4}2^{k-1}b_k^{[i]}C_i^{(1)}\le 2400\\\sum_{i=1}^{4}\sum_{k=1}^{4}2^{k-1}b_k^{[i]}\ y_{ij}R_{ij}^{*}\le N_j,\forall j=1,2,3\end{cases}\end{aligned} \tag{10}$$

(2) 简化 0-1 规划模型并化为规范形

Step1： y_{ij} 取值的讨论

由于 y_{ij} 是 0-1 变量，通过对上述模型的遍历，结合 $\sum_{j=1}^{3}y_{ij}=1,\forall i=1,2,3,4$ 和 $y_{12}=y_{13}=y_{23}=y_{41}=0$ 两个约束条件，可以得到 $\sum_{i=1}^{4}\sum_{k=1}^{4}2^{k-1}b_k^{[i]}\ y_{ij}R_{ij}^{*}\le N_j,\forall j=1,2,3$ 中的 y_{ij} 取值。得到： $y_{11}=y_{22}=y_{31}=y_{43}=1$

除上述的四个 y_{ij} 外，其余的 $y_{ij}=0$ 。代入有关的数据，给出 0-1 规划问题的规范形，见式(11):

$$\begin{aligned}&\max Z_l=\sum_{i=1}^{4}\sum_{k=1}^{4}2^{k-1}b_k^{[i]}A_i^{(1)}\\&s.t.\begin{cases}b_k^{[i]}\in\{0,1\}\\y_{11}=y_{22}=y_{31}=y_{43}=1\\y_{12}=y_{13}=y_{21}=y_{23}=y_{32}=y_{33}=y_{41}=y_{42}=0\\\sum_{i=1}^{4}\sum_{k=1}^{4}2^{k-1}b_k^{[i]}C_i^{(1)}+S_1=2400\\\sum_{i=1}^{4}\sum_{k=1}^{4}2^{k-1}b_k^{[i]}\ y_{ij}R_{ij}^{*}+S_2=N_j,\forall j=1,2,3\end{cases}\end{aligned} \tag{11}$$

其中，$S_1=(s_{11})^T$ 和 $S_2=(s_{21},s_{22},s_{23})^T$ 表示不等式化为等式约束的松弛变量。$b_k^{[i]}$ 共有 16 个，每四个对应一个 $n_i^{(1)}$ 的表示。

(3) 转化 0-1 规划模型的规范型为 QUBO 模型

由上述 0-1 规划模型的规范型，将其转化为 QUBO 模型。为了直观展示转化过程，代入 $A_i^{(1)}$ 和 y_{ij} 的具体数值如下：

$$\max Z_l=2000\sum_{k=1}^{4}b_k^{[1]}2^{k-1}+3000\sum_{k=1}^{4}b_k^{[2]}2^{k-1}+3000\sum_{k=1}^{4}b_k^{[3]}2^{k-1}+6000\sum_{k=1}^{4}b_k^{[4]}2^{k-1}$$

$$s.t.\begin{cases} b_k^{[i]}\in\{0,1\} \\ 100\sum_{k=1}^{4}b_k^{[1]}2^{k-1}+140\sum_{k=1}^{4}b_k^{[2]}2^{k-1}+200\sum_{k=1}^{4}b_k^{[3]}2^{k-1}+320\sum_{k=1}^{4}b_k^{[4]}2^{k-1}+s_{11}=2400 \\ \sum_{k=1}^{4}b_k^{[1]}2^{k-1}+s_{21}=7 \\ \sum_{k=1}^{4}b_k^{[2]}2^{k-1}+s_{22}=7 \\ \sum_{k=1}^{4}b_k^{[3]}2^{k-1}+\sum_{k=1}^{4}b_k^{[4]}2^{k-1}+s_{23}=3 \\ \sum_{k=1}^{4}2^{k-1}b_k^{[i]}\in\{0,1,2,...,14,15\} \end{cases} \tag{12}$$

$s_{11},s_{21},s_{22},s_{23}$ 为具体的松弛变量。

对 $s_{11},s_{21},s_{22},s_{23}$ 可以确定其范围：

$$\inf s\le s\le\sup s(\inf s\ge 0,\inf s\in N^*,\sup s\in N^*) \tag{13}$$

找到第一个超过 $\sup s$ 的 2^p 的最小 p 值，令其为 p_0。

$$p_0=\min p=\{2^p-\sup s\ge 0\}(p=1,2,3...) \tag{14}$$

把 p_0+1 作为松弛变量二进制的位数，得到松弛变量二进制的转化，松弛变量二进制转换的一般公式为：

$$s=\sum_{k=1}^{p_0+1}2^{k-1}b_k^{[i]}\in\{0,1,2,...,2^{p_0+1}-1\},b_k^{[i]}\in\{0,1\} \tag{15}$$

对 QUBO 模型的一般形式：$\max x_0=x^tQx\quad s.t.\,x\,binary$ 需要确定转化为二进制形式的原模型 $\max cx\quad s.t.\,Ax=b$ 中的矩阵 A：

矩阵 A 为 16×4 的矩阵，给出矩阵 A：

$$A_{16\times4}=\begin{pmatrix} 100 & 200 & 400 & 800 & 140 & 280 & 560 & 1120 & 200 & 400 & 800 & 1600 & 320 & 640 & 1280 & 2560 \\ 1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 2 & 2 & 2 & 2 & 1 & 1 & 1 & 1 \end{pmatrix}^T_{4\times16} \tag{16}$$

等式约束 $Ax=b$ 的二次惩罚项，可由下式给出：

$$P\sum\nolimits_i\left(\sum\nolimits_{j=1}^{n}a_{ij}x_{ij}-b_i\right)^2 \tag{17}$$

给出转化后的模型目标函数如下：

$$
\begin{aligned}
&2000\sum_{k=1}^{4} b_k^{[1]}2^{k-1}+3000\sum_{k=1}^{4} b_k^{[2]}2^{k-1}+3000\sum_{k=1}^{4} b_k^{[3]}2^{k-1}+6000\sum_{k=1}^{4} b_k^{[4]}2^{k-1}\\
&+P(100\sum_{k=1}^{4} b_k^{[1]}2^{k-1}+140\sum_{k=1}^{4} b_k^{[2]}2^{k-1}+200\sum_{k=1}^{4} b_k^{[3]}2^{k-1}+320\sum_{k=1}^{4} b_k^{[4]}2^{k-1}+s_{11}-2400)^2\\
&+P(\sum_{k=1}^{4} b_k^{[1]}2^{k-1}+s_{21}-7)^2 \qquad (18)\\
&+P(\sum_{k=1}^{4} b_k^{[2]}2^{k-1}+s_{22}-7)^2\\
&+P(\sum_{k=1}^{4} b_k^{[3]}2^{k-1}+\sum_{k=1}^{4} b_k^{[4]}2^{k-1}+s_{23}-3)^2
\end{aligned}
$$

由于$b_k^{[i]}\in\{0,1\}$，所以$b_k^{[i]}=\left(b_k^{[i]}\right)^2$，综上所述，给出 QUBO 模型目标函数$Z_l$的如下：

$$
\begin{aligned}
&2000\sum_{k=1}^{4} (b_k^{[1]})^2 2^{k-1}+3000\sum_{k=1}^{4} (b_k^{[2]})^2 2^{k-1}+3000\sum_{k=1}^{4} (b_k^{[3]})^2 2^{k-1}+6000\sum_{k=1}^{4} (b_k^{[4]})^2 2^{k-1}\\
&+P(100\sum_{k=1}^{4} b_k^{[1]}2^{k-1}+140\sum_{k=1}^{4} b_k^{[2]}2^{k-1}+200\sum_{k=1}^{4} b_k^{[3]}2^{k-1}+320\sum_{k=1}^{4} b_k^{[4]}2^{k-1}+s_{11}-2400)^2\\
&+P(\sum_{k=1}^{4} b_k^{[1]}2^{k-1}+s_{21}-7)^2 \qquad (19)\\
&+P(\sum_{k=1}^{4} b_k^{[2]}2^{k-1}+s_{22}-7)^2\\
&+P(\sum_{k=1}^{4} b_k^{[3]}2^{k-1}+\sum_{k=1}^{4} b_k^{[4]}2^{k-1}+s_{23}-3)^2
\end{aligned}
$$

给定具体惩罚项P值（相对较大的数），得到具体 QUBO 模型，式(20)是一个二次型，将二次型转化为矩阵形式，得到 QUBO 模型的矩阵Q。此处Q过大（一个 16×16 的矩阵），此处的二次型过于复杂，在附件中给出，现给出形式上的 QUBO 模型表达式：

$$
\begin{aligned}
&QUBO:\max Z_l=b_k^{[i]}Qb_k^{[i]T}\\
&s.t.\, b_k^{[i]}\ binary
\end{aligned} \qquad (20)
$$

至此，原模型转化为 QUBO 模型，考虑采用 Kaiwu SDK 内置的模拟退火求解器和 CIM 求解器进行 QUBO 模型的求解。

6.1.3 QUBO 模型的求解

分别采用模拟退火算法，以及 CIM 模拟器对 QUBO 模型进行求解，并对两种方法的求解结果进行对照验证，最终确定模型的最优方案。具体步骤、结果如下：

(1) 基于模拟退火算法求解 QUBO 模型

模拟退火是一种优化算法，其模拟了固体物质在高温下冷却的过程，通过随机扰动和接受“劣质解”的策略来逐步收敛到全局最优解。

在算法启动时，需要初始化模型的参数，包括初始温度T_0，最终停止温度T_f，降温速度参数$alpha$，控制参数k，控制变量k的增加量$delate$，以及 QUBO 模型的初始解，逐步迭代，每次计算新解集的目标值$f(x_1)$，若新目标值小于原目标值$f(x_0)$，则记录该解作为当前最优解，若$f(x_1)>f(x_0)$，则以概率P记录该解为最优解，计算当前温度T，以及迭代次数k一直到温度达到停止温度，算法停止，P的计算见下式：

$$
P=e^x,\lambda=\frac{f(x_0)f(x)}{k} \qquad (21)
$$

模拟退火伪代码如下：

算法 1 模拟退火算法

1.**procedure** ANNEALING APPROACH

2. $X_0, T \leftarrow \text{Initializa(QUBO)}, T_0$

3. $alpha, delta, k, T, T_f \leftarrow alpha_0, delta_0, k_0, T_0, T_f$

4. **while** $T > T_f$

5. $X_1 \leftarrow Optimize(QUBO)$

6. $df \leftarrow f(X_1) - f(X_2)$

7. ***if*** $df < 0$ ***or*** $random(0,1) < \exp\left(\frac{df}{k \cdot T}\right)$

8. $X_0 \leftarrow X_1$

9. $T \leftarrow T * alpha$

10. $k \leftarrow k + delta$

11.***return*** X_0

(2) 基于 CIM 模拟器求解 QUBO 模型

CIM 是指量子计算机中的注入同步激光伊辛机（injection synchronous laser Ising machine）的缩写。它是一种基于光纤中的激光脉冲作为计算的基本单元的计算机，也被称为量子比特（qubits）。

本文基于 Kaiwa SDK 的 CIM 模拟器对模型进行求解，具体使用步骤如下图：

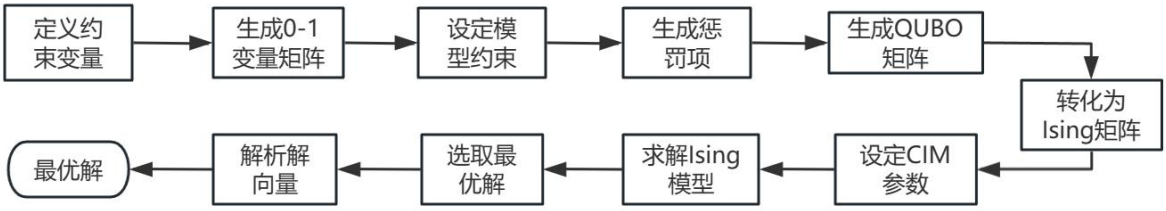

图 4 CIM 模型求解步骤

(3) 求解结果及分析

模拟退火算法的初始化参数如下：初始温度 100，降温系数 0.99，截止温度 0.001，温度迭代深度 10；**CIM 模拟器**初始化参数如下：输入相位 0，泵浦功率 0.7，噪声强度 0.01，每次运行的圈数 1000，每全的时间步长 0.1。

记最优解矩阵 $B_{1\times16} = (B_k^{[1]} \mid B_k^{[2]} \mid B_k^{[3]} \mid B_k^{[4]}), B_k^{[i]} = (b_1^{[i]} \mid b_2^{[i]} \mid b_3^{[i]} \mid b_4^{[i]})$，则 B 是由 16 个 0-1 变量 $b_k^{[i]}$ 构成的 1×16 的矩阵。最终两种方法求得 QUBO 模型的最优解矩阵一致，并且有两个可行解：

$$\begin{cases} B_1 = \{0,0,0,0,0,1,1,0,0,0,1,1,0,0,1,1\} \\ B_2 = \{0,1,1,1,0,1,1,1,0,0,1,0,0,0,0,1\} \end{cases} \tag{22}$$

其中，前 4 位二进制数转化为 10 进制数表示挖 1 采购数量，其他挖掘机采购数量以此类推，最后得到四个挖掘机的采购数量，最终最优采购方案如下表：

表 2 长期利润下最优挖掘机采购方案表

	挖 1	挖 2	挖 3	挖 4	总利润
最优解 1	0	6	3	3	51000
最优解 2	7	7	2	1	51000

最优采购方案 1 为采购 1 型挖掘机 0 辆，2 型挖掘机 6 辆，3 型挖掘机 3 辆，4 型挖掘机 3 辆；**最优采购方案 2** 为采购 1 型挖掘机 7 辆，2 型挖掘机 7 辆，3 型挖掘机 2 辆，4 型挖掘机 1 辆，最大利润均为 **51000 万元**。

6.2 问题二模型的建立与求解

为解决挖掘机和矿车固定 5 年寿命的情况下的总利润最大化的问题，要求给出挖掘机和矿车之间的匹配关系以及采购的挖掘机型号及其对应数量。决策变量为采购挖掘机的型号及其对应数量，目标函数为总利润（利润=收益-成本）的最大化。考虑题目中的有关约束条件，构建优化模型。然后将该模型转化为 QUBO 模型。最后利用 Kaiwu SDK 内置的模拟退火求解器和 CIM 求解器进行求解。

6.2.1 五年内利润最大的优化模型建立

(1) 决策变量

决策变量 $1, n_i^{(1)}(i=1,2,3,4)$：第 i 型挖掘机的采购量（单位：台）。

决策变量 $2, x_i^{(1)} \in \{0,1\}$：第 i 型挖掘机是否采购，其中 $x_i^{(1)}=0$ 代表不采购，$x_i^{(1)}=1$ 代表采购（单位：/）。

(2) 目标函数

由于目标函数为总利润（利润=收益-成本）最大，分成子模块逐一给出成本和收益。成本：

• 人工成本（单位：万元）

$$op_1 = 0.006\sum_{i=1}^{4} P_i^{(1)} n_i^{(1)} \tag{23}$$

$$op_2 = 0.006\sum_{j=1}^{3} P_j^{(2)} n_j^{(2)} \tag{24}$$

其中，op_1 表示挖掘机 5 年的总人工成本，op_2 表示矿车 5 年的总人工成本，$P_i^{(1)}$ 表示第 i 型挖掘机的人工成本，$P_j^{(2)}$ 表示第 j 型矿车的人工成本，$n_i^{(1)}$ 表示第 i 型挖掘机的购买数量，$n_j^{(1)}$ 表示第 j 型矿车的使用数量。题中给出的人工成本单位为元/月，换算单位为万元，故有系数 0.006.

• 维护成本（单位：万元）

$$op_3 = 0.006\sum_{i=1}^{4} M_i^{(1)} n_i^{(1)} \tag{25}$$

$$op_4 = 0.006\sum_{j=1}^{3} M_j^{(2)} n_j^{(2)} \tag{26}$$

其中，op_3 表示挖掘机 5 年的总维护成本，op_4 表示矿车 5 年的总维护成本，$M_i^{(1)}$ 表示第 i 型挖掘机的维护成本，$M_j^{(2)}$ 表示第 j 型矿车的维护成本。题中给出的维护成本单位为元/月，换算单位为万元，故有系数 0.006.

• 采购费用成本（单位：万元）

$$op_5 = \sum_{i=1}^{4} C_i^{(1)} n_i^{(1)} \tag{27}$$

其中，op_5 表示挖掘机的总采购费用，$C_i^{(1)}$ 表示第 i 型挖掘机的采购费用。

• 油耗费用成本（单位：万元）

$$op_6 = 6.72\sum_{i=1}^{4} O_i^{(1)} n_i^{(1)} \tag{28}$$

$$op_7 = 6.72\sum_{j=1}^{3} O_j^{(2)} n_j^{(2)} \tag{29}$$

其中，op_6 表示挖掘机的总油耗费用，op_7 表示矿车的总油耗费用，$O_i^{(1)}$ 表示第 i 型挖掘机的油耗费用，$O_j^{(2)}$ 表示第 j 型矿车的维护成本。题中给出的油耗费用单位为元/升，故有系数 6.72.

收益：

• 挖矿收益（单位：万元）

由题意，挖矿带来的收益等于五年的挖矿总量乘以矿价，矿价为 20 元/立方米，下面计算五年的挖矿总量 M_5。

$$M_5 = 9600\sum_{i=1}^{4}\left(E_i V_i n_i^{(1)}\left(\sum_{j=1}^{3}\frac{R_{ij}}{R_{ij}^*} y_{ij}\right)\right) \tag{30}$$

其中，E_i 表示第 i 型挖掘机的作业效率，V_i 表示第 i 型挖掘机的斗容，$R_{ij} \in Z^+$ 表示第 i 型挖掘机实际情况下匹配到第 j 型矿车的数目，$R_{ij}^* \in Z^+$ 则表示第 i 型挖掘机理想情况下匹配到第 j 型矿车的数目。

因此 $\frac{R_{ij}}{R_{ij}^*}$ 表示实际挖掘机效率（根据题目中的假设：若挖掘机与矿车匹配关系恰好，每日作业量达到挖掘机效率，效率为 100%；若给挖掘机分配的矿车少，每日作业量则达到挖掘机效率乘以比例 $\frac{R_{ij}}{R_{ij}^*}$），y_{ij} 表示第 i 型挖掘机是否与第 j 型矿车匹配（若匹配，则 $y_{ij} = 1$，否则 $y_{ij} = 0$）。

得到挖矿收益 ers 为 $20\times M_5$,即：

$$ers = 19.2\sum_{i=1}^{4}\left(E_i V_i n_i^{(1)}\left(\sum_{j=1}^{3}\frac{R_{ij}}{R_{ij}^*} y_{ij}\right)\right) \tag{31}$$

题中给出的矿石价格单位为元/立方米，故有系数 19.2.

基于启动资金及上述成本与收益的推导分析，得到目标函数如下：

$$\max Z = ers - \sum_{k=1}^{7} op_k \tag{32}$$

(3) 约束条件

约束条件 1：启动资金预算约束。一开始采购挖掘机时要考虑资金约束。

$$\sum_{i=1}^{4} C_i^{(1)} n_i^{(1)} \le 2400 \tag{33}$$

约束条件 2：矿车运营约束。出于对实际运营考量，整体包含的挖掘机型号不能少于三种。

$$\sum_{i=1}^{4} x_i^{(1)} \ge 3, x_i^{(1)} \in \quad \} \tag{34}$$

约束条件 3 挖掘机和矿车的匹配约束。在挖掘机作业中，考虑到挖掘机铲斗宽度和矿车宽度的对应关系以及装车效率，需要满足特定的挖掘机与矿车的匹配关系（题中已给出）。基于给出的矿车及挖掘机型号匹配绑定假设，即每个型号只对应一种矿车，对

此我们引入一个 0-1 变量 $y_{ij}(i=1,2,3,4;j=1,2,3)$。$y_{ij}$ 表示第 j 型矿车是否与第 i 型挖掘机匹配。1 为匹配。0 为不匹配。结合题中数据给出以下约束：

$$\begin{cases}\sum_{j=1}^{3} y_{ij}=1,\forall i=1,2,3,4,y_{ij}\in\{0,1\}\\ y_{12}=y_{13}=y_{23}=y_{41}=0\end{cases} \tag{35}$$

其中，由于同一类挖掘机可以和不同类型的矿车进行匹配，但是匹配关系固定不变，而且同一型号挖掘机只能匹配同一型号的矿车，所以可以得到 $\sum_{j=1}^{3} y_{ij}=1,\forall i=1,2,3,4$。且不能匹配的挖掘机和矿车的 y_{ij} 必须为 0.

约束条件 4：矿车实际使用数量约束。N=(7,7,3)表示矿车总数量，矿车实际使用数量约束如下：

$$n_j^{(2)}\le N_j,\forall j=1,2,3 \tag{36}$$

约束条件 5：挖掘机与矿车匹配后的矿车数量约束。

R_{ij}^* 则表示 i 型挖掘机理想情况下匹配 j 型矿车的数目。y_{ij} 表示 i 型挖掘机是否与 j 型矿车匹配（若匹配，则 $y_{ij}=1$，否则 $y_{ij}=0$），效率约束如下：

$$\max \overline{Z}=\overline{ers}-\sum_{k=1}^{7}\overline{op}_k \tag{37}$$

$$s.t.\begin{cases}\overline{ers}=19.2\sum_{i=1}^{10}\left(E_iV_in_i^{(1)}\left(\sum_{j=1}^{10}\frac{R_{ij}}{R_{ij}^*}y_{ij}\right)\right)\\ \overline{op_1}=0.006\sum_{i=1}^{10}P_i^{(1)}n_i^{(1)}\\ \overline{op_2}=0.006\sum_{j=1}^{10}P_j^{(2)}n_j^{(2)}\\ \overline{op_3}=0.006\sum_{i=1}^{10}M_i^{(1)}n_i^{(1)}\\ \overline{op_4}=0.006\sum_{j=1}^{10}M_j^{(2)}n_j^{(2)}\\ \overline{op_5}=\sum_{i=1}^{10}C_i^{(1)}n_i^{(1)}\\ \overline{op_6}=6.72\sum_{i=1}^{10}O_i^{(1)}n_i^{(1)}\\ \overline{op_7}=6.72\sum_{j=1}^{10}O_j^{(2)}n_j^{(2)}\end{cases}$$

约束条件 6：挖掘机效率比例约束。第 i 型挖掘机理想情况下匹配到第 j 型矿车的数目要大于等于 i 型挖掘机实际情况下匹配到 j 型矿车的数目。

$$R_{ij}\le R_{ij}^* \tag{38}$$

约束条件 7：挖掘机购买数量的整数约束。由于挖掘机的购买数量为整数，约束如下：

$$n_i^{(1)}\in\mathbb{Z}^+(i=1,2,3,4) \tag{39}$$

约束条件 8：矿车数量的整数约束，矿车的数量为整数，约束如下：

$$n_j^{(2)} \in \mathbb{Z}^+ (j=1,2,3) \tag{40}$$

综上所述，可以构建 5 年总利润最大的优化模型如下，见式(41):

$$\max \overline{Z} = \overline{ers} - \sum_{k=1}^{7} \overline{op}_k \tag{41}$$

$$s.t. \begin{cases} \overline{ers} = 19.2 \sum_{i=1}^{10} \left(E_i V_i n_i^{(1)} \left(\sum_{j=1}^{10} \frac{R_{ij}}{R_{ij}^*} y_{ij} \right) \right) \\ \overline{op_1} = 0.006 \sum_{i=1}^{10} P_i^{(1)} n_i^{(1)} \\ \overline{op_2} = 0.006 \sum_{j=1}^{10} P_j^{(2)} n_j^{(2)} \\ \overline{op_3} = 0.006 \sum_{i=1}^{10} M_i^{(1)} n_i^{(1)} \\ \overline{op_4} = 0.006 \sum_{j=1}^{10} M_j^{(2)} n_j^{(2)} \\ \overline{op_5} = \sum_{i=1}^{10} C_i^{(1)} n_i^{(1)} \\ \overline{op_6} = 6.72 \sum_{i=1}^{10} O_i^{(1)} n_i^{(1)} \\ \overline{op_7} = 6.72 \sum_{j=1}^{10} O_j^{(2)} n_j^{(2)} \end{cases}$$

6.2.2 QUBO 模型的转化

在 6.2.1 中，已经建立了 5 年内利润最大的优化模型，现在需要将该模型转化为 QUBO 模型的形式，便于使用 Kaiwu SDK 中的内置求解器进行求解。

(1) 转化上述原始模型为 0-1 规划模型

Step1：将 $n_i^{(1)} x_i^{(1)}$ 合并为单一的变量 $n_i^{(1)}$

第 i 型挖掘机的采购量 $n_i^{(1)}$ 和第 i 型挖掘机是否采购 $x_i^{(1)}$ 可以用 $n_i^{(1)}$ 进行统一，即 $n_i^{(1)} = 0$ 时表示第 i 型挖掘机不采购。统一之后可以去掉 $x_i^{(1)}$，$n_i^{(1)} \in \mathbb{Z}^+ \cap 0$。

Step2：对 $n_i^{(1)}$ 的取值进行估测并对其进行二进制转化

根据约束条件 $\sum_{i=1}^{4} C_i^{(1)} n_i^{(1)} \le 2400$，可以估测出 $n_i^{(1)} \in \{0,1,2,...,14,15\}$。

任意 $n_i^{(1)}$，可用四个 0-1 变量表示，例: $n_1^{(1)} = b_1^{[1]} + 2b_2^{[1]} + 4b_3^{[1]} + 8b_4^{[1]}, b_k^{[1]}\ binary$。可知如下二进制转换公式：

$$n_i^{(1)} = \sum_{k=1}^{4} 2^{k-1} b_k^{[i]} \in \{0,1,2,...,14,15\}, b_k^{[i]} \in \{0,1\} \tag{42}$$

经过转换后，将原优化问题的决策变量转化为 $b_k^{[i]}\ binary$，这是一个一般的 0-1 规划问题，写出该 0-1 规划问题如下：

$$
\max Z = ers - \sum_{k=1}^{7} op_k
$$

$$
s.t. \begin{cases}
b_k^{[i]} \in \{0,1\} \\
y_{ij} \in \{0,1\} \\
R_{ij} \in \mathbb{Z}^+ (i=1,2,3,4; j=1,2,3) \\
R_{ij}^* \in \mathbb{Z}^+ (i=1,2,3,4; j=1,2,3) \\
\sum_{j=1}^{3} y_{ij} = 1, \forall i = 1,2,3,4; y_{12} = y_{13} = y_{23} = y_{41} = 0 \\
\sum_{k=1}^{4} 2^{k-1} b_k^{[i]} \in Z^+ (i=1,2,3,4), n_j^{(2)} \in Z^+ (j=1,2,3) \\
\sum_{i=1}^{4} \sum_{k=1}^{4} 2^{k-1} b_k^{[i]} C_i^{[1]} \le 2400 \\
\sum_{i=1}^{4} \sum_{k=1}^{4} 2^{k-1} b_k^{[i]} y_{ij} R_{ij}^* \le N_j, \forall j = 1,2,3 \\
n_j^{(2)} \le N_j, \forall j = 1,2,3 \\
R_{ij} \le R_{ij}^*
\end{cases} \tag{43}
$$

(2) 简化 0-1 规划模型并化为规范形

Step1： y_{ij}取值的讨论

由于 y_{ij} 是 0-1 变量，通过对上述模型的遍历，结合 $\sum_{j=1}^{3} y_{ij} = 1, \forall i = 1,2,3,4$ 和 $y_{12} = y_{13} = y_{23} = y_{41} = 0$ 两个约束条件，可以得到 $\sum_{i=1}^{4} \sum_{k=1}^{4} 2^{k-1} b_k^{[i]}\ y_{ij} R_{ij}^* \le N_j, \forall j = 1,2,3$ 中的 y_{ij} 取值。得到：$y_{11} = y_{22} = y_{33} = y_{43} = 1$

除上述的四个 y_{ij} 外，其余的 $y_{ij} = 0$。可以代入有关数据.

Step2： R_{ij} 和 $n_j^{(2)}$ 取值的讨论

由于题目中要求盈利最大化，所以 $n_j^{(2)} = N_j, \forall j = 1,2,3$，$R_{ij} = R_{ij}^*$ 至此，可以写出此 0-1 规划问题的规范型（具体写出目标函数），S 为松弛变量。

$$
\begin{aligned}
\max Z &= ers - \sum_{k=1}^{7} op_k \\
&= 19200\sum_{i=1}^{4}\left(E_i V_i \sum_{k=1}^{4} 2^{k-1} b_k^{[i]} \left(\sum_{j=1}^{3} y_{ij}\right)\right) - \sum_{i=1}^{4} C_i^{(1)} \sum_{k=1}^{4} 2^{k-1} b_k^{[i]} \\
&\quad - 0.006\sum_{i=1}^{4} P_i^{(1)} \sum_{k=1}^{4} 2^{k-1} b_k^{[i]} - 0.006\sum_{i=1}^{4} M_i^{(1)} \sum_{k=1}^{4} 2^{k-1} b_k^{[i]} - 6.72\sum_{i=1}^{4} O_i^{(1)} \sum_{k=1}^{4} 2^{k-1} b_k^{[i]} \\
&\quad - 0.006\sum_{j=1}^{3} P_j^{(2)} N_j - 0.006\sum_{j=1}^{3} M_j^{(2)} N_j - 6.72\sum_{j=1}^{3} O_j^{(2)} N_j
\end{aligned}
$$

$$
s.t.\begin{cases}
b_k^{[i]} \in \{0,1\} \\
y_{11} = y_{22} = y_{33} = y_{43} = 1 \\
y_{12} = y_{13} = y_{21} = y_{23} = y_{32} = y_{33} = y_{41} = y_{42} = 0 \\
\sum_{k=1}^{4} 2^{k-1} b_k^{[i]} \in Z^+ (i = 1,2,3,4), n_j^{(2)} \in Z^+ (j = 1,2,3) \\
\sum_{i=1}^{4}\sum_{k=1}^{4} 2^{k-1} b_k^{[i]} C_i^{[1]} + S_1 = 2400 \\
\sum_{i=1}^{4}\sum_{k=1}^{4} 2^{k-1} b_k^{[i]} y_{ij} R_{ij}^* + S_2 = N_j, \forall j = 1,2,3 \\
n_j^{(2)} = N_j, \forall j = 1,2,3 \\
R_{ij} = R_{ij}^*
\end{cases} \tag{44}
$$

(3) 转化 0-1 规划模型的规范型为 QUBO 模型

由上述 0-1 规划模型的规范型，根据附件 1 中的有关内容将其转化为 QUBO 模型，首先将 0-1 规划模型规范型展开：

$$
\max Z = ers - \sum_{k=1}^{7} op_k
$$

$$
s.t.\begin{cases}
b_k^{[i]} \in \{0,1\} \\
100\sum_{k=1}^{4} b_k^{[1]} 2^{k-1} + 140\sum_{k=1}^{4} b_k^{[2]} 2^{k-1} + 200\sum_{k=1}^{4} b_k^{[3]} 2^{k-1} + 320\sum_{k=1}^{4} b_k^{[4]} 2^{k-1} + s_{11} = 2400 \\
\sum_{k=1}^{4} b_k^{[1]} 2^{k-1} R_{ij}^* + s_{21} = 7 \\
\sum_{k=1}^{4} b_k^{[2]} 2^{k-1} R_{ij}^* + s_{22} = 7 \\
\sum_{k=1}^{4} b_k^{[3]} 2^{k-1} R_{ij}^* + \sum_{k=1}^{4} b_k^{[4]} 2^{k-1} R_{ij}^* + s_{23} = 3
\end{cases} \tag{45}
$$

松弛变量二进制转换的一般公式为：

$$
s = \sum_{k=1}^{p_0+1} 2^{k-1} b_k^{[i]} \in \{0,1,2,...,2^{p_0+1}-1\}, b_k^{[i]} \in \{0,1\} \tag{46}
$$

对 QUBO 模型的一般形式：$\max x_0 = x^t Qx \quad s.t.\, x\, binary$ 需要确定转化为二进制形式的原模型 $\max cx \quad s.t.\, Ax = b$ 中的矩阵 A（见附件），等式约束 $Ax = b$ 的二次惩罚项，可由下式给出：

$$P\sum_i\left(\sum_{j=1}^{n} a_{ij}x_{ij}-b_i\right)^2 \tag{47}$$

由于$b_k^{[i]}$是 0-1 变量，所以$b_k^{[i]}=\left(b_k^{[i]}\right)^2$，综上所述，给出 QUBO 模型目标函数$Z$如下：

$$\begin{aligned}
\max \quad &= 19.2\sum_{i=1}^{4}\left(E_iV_i\sum_{k=1}^{4}2^{k-1}\left(b_k^{[i]}\right)^2\left(\sum_{j=1}^{3}y_{ij}\right)\right)-\sum_{i=1}^{4}C_i^{(1)}\sum_{k=1}^{4}2^{k-1}\left(b_k^{[i]}\right)^2\\
&-0.006\sum_{i=1}^{4}P_i^{(1)}\sum_{k=1}^{4}2^{k-1}\left(b_k^{[i]}\right)^2-0.006\sum_{i=1}^{4}M_i^{(1)}\sum_{k=1}^{4}2^{k-1}\left(b_k^{[i]}\right)^2-6.72\sum_{i=1}^{4}O_i^{(1)}\sum_{k=1}^{4}2^{k-1}\left(b_k^{[i]}\right)^2\\
&-0.006\sum_{j=1}^{3}P_j^{(2)}N_j-0.006\sum_{j=1}^{3}M_j^{(2)}N_j-6.72\sum_{j=1}^{3}O_j^{(2)}N_j\\
&+P(100\sum_{k=1}^{4}b_k^{[1]}2^{k-1}+140\sum_{k=1}^{4}b_k^{[2]}2^{k-1}+200\sum_{k=1}^{4}b_k^{[3]}2^{k-1}+320\sum_{k=1}^{4}b_k^{[4]}2^{k-1}+s_{11}-2400)^2\\
&+P(\sum_{k=1}^{4}b_k^{[1]}2^{k-1}R_{ij}^{*}+s_{21}-7)^2+P(\sum_{k=1}^{4}b_k^{[2]}2^{k-1}R_{ij}^{*}+s_{22}-7)^2\\
&+P(\sum_{k=1}^{4}b_k^{[3]}2^{k-1}R_{ij}^{*}+\sum_{k=1}^{4}b_k^{[4]}2^{k-1}R_{ij}^{*}+s_{23}-3)^2
\end{aligned} \tag{48}$$

给定具体惩罚项P值（相对较大的数），得到具体 QUBO 模型，式(48)是一个二次型，将二次型转化为矩阵形式，得到 QUBO 模型的矩阵Q。此处Q过大（一个 16×16 的矩阵），此处的二次型过于复杂，在附件中给出，给出形式上的 QUBO 模型表达式：

$$\begin{aligned}
&QUBO:\max Z_l=b_k^{[i]}Qb_k^{[i]T}\\
&s.t.b_k^{[i]}\ \ binary
\end{aligned} \tag{49}$$

6.2.3 QUBO 模型的求解

对问题二 QUBO 模型的求解，依旧使用模拟退火算法求解器以及 CIM 模拟器两种方法求解，最终求解结果如下表：

表 3 最大化利润下的挖掘机采购方案表

方法	挖 1	挖 2	挖 3	挖 4	总利润（万元）
模拟退火	7	7	3	0	60193.73
CIM	7	7	2	1	60392.45

表 4 挖掘机与矿车的匹配关系表(模拟退火求解)

	挖 1	挖 2	挖 3	挖 4
矿 1	1	\	\	\
矿 2	\	1	\	\
矿 3	\	\	1	\

表 5 挖掘机与矿车的匹配关系表(CIM 求解)

	挖 1	挖 2	挖 3	挖 4
矿 1	1	\	\	\
矿 2	\	1	\	\
矿 3	\	\	1	1

在最优解中，可以看到模拟退火求解器的最优解小于 CIM 模拟器的最优解，这是由于模拟退火算法为启发式算法，算法本身加入了概率项，因此在一定的迭代次数下，依旧有概率陷入局部最优解。由此我们可以认为 CIM 模拟器的求解答案更为准确，且 CIM 模拟器的求解稳定性很好，明显好于传统模拟退火算法。

基于 CIM 模拟器求解，绘制了量子比特求解演化图以及哈密顿量随迭代次数的收敛图如下：

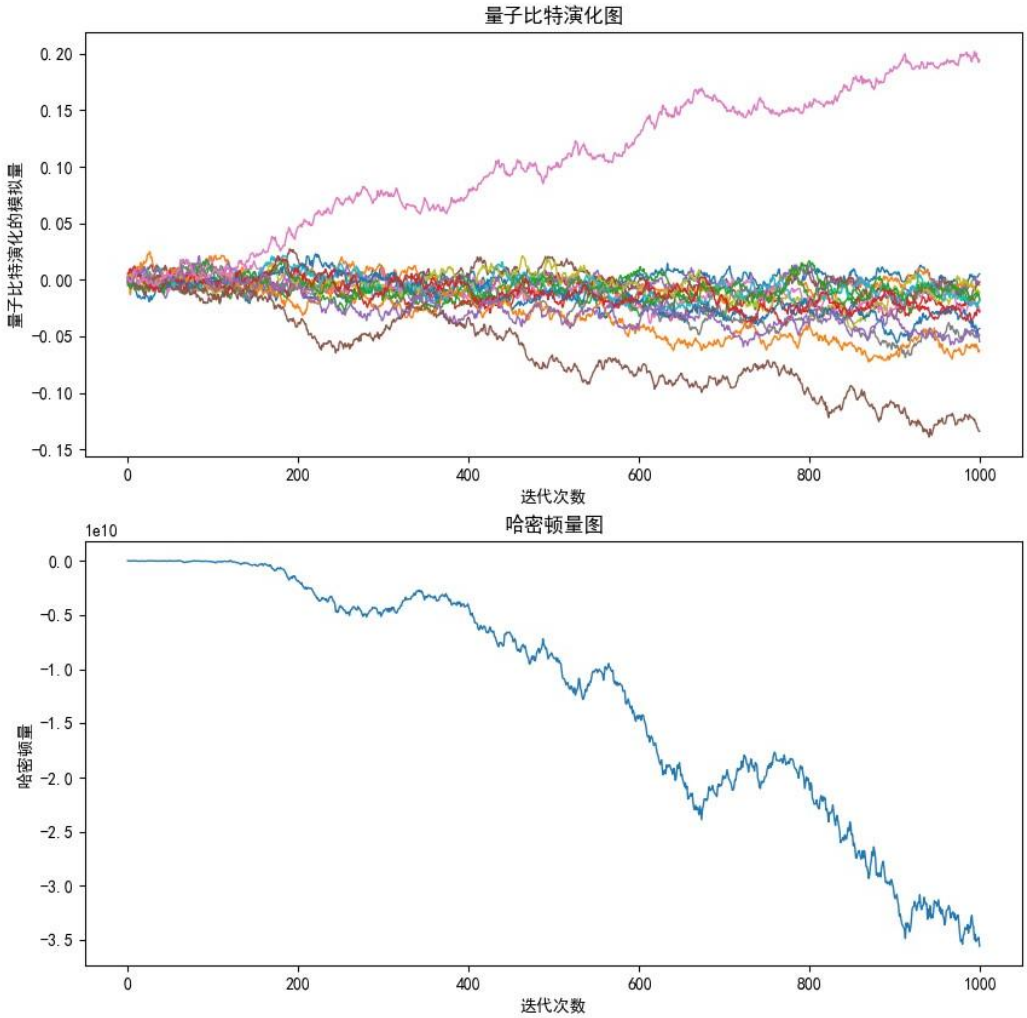

图 5 问题二 CIM 模拟器的求解迭代图

由上图可见，量子比特模拟量演化过程接近理论预期，并且哈密顿量值整体上在迭代过程保持递减趋势，随着迭代过程逐渐趋于最优解，因此可见 CIM 模拟器的求解效率，以及对于量子比特数较多的问题的求解具有较好效果。

6.3 问题三模型的建立与求解

为解决挖掘机和矿车固定 5 年寿命的情况下的总利润最大化的问题，要求给出挖掘机和矿车之间的匹配关系以及采购的挖掘机型号及其对应数量。决策变量为采购挖掘机的型号及其对应数量，目标函数为总利润最大化，构建优化模型。然后将该模型转化为 QUBO 模型。最后利用 Kaiwu SDK 内置的求解器进行求解。

问题三实际上是问题二的推广形式，增加了可选挖掘机的数量、可选矿车的数量及其匹配关系等条件，同时要求对挖掘机的种类不少于 5 种，且启动资金为 4000 万元。在第二问模型的基础之上，需要修改部分的约束条件，以建立原始优化模型。

6.3.1 五年内利润最大的优化模型重新建立

相较于第二问建立的模型，增加了挖掘机的类型、矿车类型及其匹配关系等条件。要求对挖掘机的种类不少于 5 种，且启动资金为 4000 万元。决策变量不变，需要对目标函数和约束条件进行修改。

(1) 目标函数的修改

由于挖掘机的类型、矿车的类型的变化，需要修改人工成本、维护成本、采购费用、油耗费用、挖矿收益。

$$\max \overline{Z} = \overline{ers} - \sum_{k=1}^{7} \overline{op}_k$$

$$s.t. \begin{cases} \overline{ers} = 19.2\sum_{i=1}^{10}\left(E_i V_i n_i^{(1)} \left(\sum_{j=1}^{10} \frac{R_{ij}}{R_{ij}^*} y_{ij} \right) \right) \\ \overline{op_1} = 0.006\sum_{i=1}^{10} P_i^{(1)} n_i^{(1)} \\ \overline{op_2} = 0.006\sum_{j=1}^{10} P_j^{(2)} n_j^{(2)} \\ \overline{op_3} = 0.006\sum_{i=1}^{10} M_i^{(1)} n_i^{(1)} \\ \overline{op_4} = 0.006\sum_{j=1}^{10} M_j^{(2)} n_j^{(2)} \\ \overline{op_5} = \sum_{i=1}^{10} C_i^{(1)} n_i^{(1)} \\ \overline{op_6} = 6.72\sum_{i=1}^{10} O_i^{(1)} n_i^{(1)} \\ \overline{op_7} = 6.72\sum_{j=1}^{10} O_j^{(2)} n_j^{(2)} \end{cases} \tag{50}$$

(2) 约束条件的修改

挖掘机的种类不少于 5 种，启动资金为 4000 万元。

主要修改启动资金预算约束和矿车运营约束，以及涉及到挖掘机的类型、矿车的类型的约束也需要改动。

$$\begin{cases} \sum_{i=1}^{10} C_i^{(1)} n_i^{(1)} \le 4000 \\ \sum_{i=1}^{10} x_i^{(1)} \ge 5, x_i^{(1)} \in \{0,1\} \end{cases} \tag{51}$$

已购买的矿车数量 N_j 也有变化，可以给出：

$$N_j = (5,5,5,5,5,3,3,3,3,3) \tag{52}$$

在问题三中可选挖掘机的型号由 4 种变为 10 种，i 变为 $i = \{1,2,3,4,5,6,7,8,9,10\}$。在问题三中可选矿车的型号由 3 种变为 10 种，j 变为 $j = \{1,2,3,4,5,6,7,8,9,10\}$。

综上，问题三模型是问题二的推广，直接给出问题三的优化模型：

$$
\max \overline{Z} = \overline{ers} - \sum_{k=1}^{7} \overline{op}_k
$$

$$
s.t. \begin{cases}
x_i^{(1)} \in \{0,1\}, y_{ij} \in \{0,1\} \\
n_i^{(1)} \in Z^+ (i = 1,2,...,10), n_j^{(2)} \in Z^+ (j = 1,2,...,10) \\
\sum_{j=1}^{10} y_{ij} = 1, \forall i = 1,2,...,10; \\
\sum_{k=5}^{10} y_{1k} = 0, \sum_{k=6}^{10} y_{2k} = 0, \sum_{k=8}^{10} y_{3k} = 0, \sum_{k}^{1,9,10} y_{4k} = 0, \sum_{k}^{1,2,10} y_{5k} = 0 \\
\sum_{k=1}^{3} y_{6k} = 0, \sum_{k=1}^{4} y_{7k} = 0, \sum_{k=1}^{4} y_{8k} = 0, \sum_{k=1}^{5} y_{9k} = 0, \sum_{k=1}^{6} y_{10k} = 0 \\
\sum_{i=1}^{10} C_i^{(1)} n_i^{(1)} \le 4000, \sum_{i=1}^{10} x_i^{(1)} \ge 5 \\
n_j^{(2)} \le N_j, \forall j = 1,2,...,10, R_{ij} \le R_{ij}^* \\
\sum_{i=1}^{10} n_i^{(1)} y_{ij} R_{ij}^* \le N_j, \forall j = 1,2,...,10
\end{cases} \tag{53}
$$

相应的 $\overline{ers}$ 和 $\overline{op}_k (k = 1,2,...,7)$：

$$
\begin{cases}
\overline{op_1} = 0.006 \sum_{i=1}^{10} P_i^{(1)} n_i^{(1)} \\
\overline{op_2} = 0.006 \sum_{j=1}^{10} P_j^{(2)} n_j^{(2)} \\
\overline{op_3} = 0.006 \sum_{i=1}^{10} M_i^{(1)} n_i^{(1)} \\
\overline{op_4} = 0.006 \sum_{j=1}^{10} M_j^{(2)} n_j^{(2)} \\
\overline{op_5} = \sum_{i=1}^{10} C_i^{(1)} n_i^{(1)} \\
\overline{op_6} = 6.72 \sum_{i=1}^{10} O_i^{(1)} n_i^{(1)} \\
\overline{op_7} = 6.72 \sum_{j=1}^{10} O_j^{(2)} n_j^{(2)} \\
\overline{ers} = 19.2 \sum_{i=1}^{10} \left(E_i V_i n_i^{(1)} \left(\sum_{j=1}^{10} \frac{R_{ij}}{R_{ij}^*} y_{ij} \right) \right)
\end{cases} \tag{54}
$$

6.3.2 QUBO 模型的转化

在 6.3.1 中，已经建立了 5 年内利润最大的优化模型，现在需要将该模型转化为 QUBO 模型的形式，便于使用 Kaiwu SDK 中的内置求解器进行求解。

(1) 转化上述原始模型为 0-1 规划模型

Step1：将 $n_i^{(1)} x_i^{(1)}$ 合并为单一的变量 $n_i^{(1)}$

第 i 型挖掘机的采购量 $n_i^{(1)}$ 和第 i 型挖掘机是否采购 $x_i^{(1)}$ 可以用 $n_i^{(1)}$ 进行统一，即 $n_i^{(1)} = 0$ 时表示第 i 型挖掘机不采购。统一之后可以去掉 $x_i^{(1)}$，$n_i^{(1)} \in \mathbb{Z}^+ \cap 0$。

Step2：对 $n_i^{(1)}$ 的取值进行估测并对其进行二进制转化

根据约束条件 $\sum_{i=1}^{4} C_i^{(1)} n_i^{(1)} \le 4000$，可以估测出 $n_i^{(1)} \in \{0,1,2,...,41,42\}$。

对于任意的 $n_i^{(1)}$，我们可以采用六个 0-1 变量 $b_k^{[i]} \in \{0,1\}$ 进行表示，例如: $n_1^{(1)} = b_1^{[1]} + 2b_2^{[1]} + 4b_3^{[1]} + 8b_4^{[1]} + 16b_5^{[1]} + 32b_6^{[1]}, b_k^{[1]}\ binary$。可知如下二进制转换公式：

$$n_i^{(1)} = \sum_{k=1}^{4} 2^{k-1} b_k^{[i]} \in \{0,1,2,...,62,63\}, b_k^{[i]} \in \{0,1\} \tag{55}$$

经过转换后，将原优化问题的决策变量转化为 $b_k^{[i]} \in \{0,1\}$，这是一个一般的 0-1 规划问题，写出该 0-1 规划问题如下：

$$\max \overline{Z} = \overline{ers} - \sum_{k=1}^{7} \overline{op}_k$$

$$s.t. \begin{cases} x_i^{(1)} \in \{0,1\}, y_{ij} \in \{0,1\} \\ n_i^{(1)} \in Z^+ (i = 1,2,...,10), n_j^{(2)} \in Z^+ (j = 1,2,...,10) \\ \sum_{j=1}^{10} y_{ij} = 1, \forall i = 1,2,...,10; \\ \sum_{k=5}^{10} y_{1k} = 0, \sum_{k=6}^{10} y_{2k} = 0, \sum_{k=8}^{10} y_{3k} = 0, \sum_{k}^{1,9,10} y_{4k} = 0, \sum_{k}^{1,2,10} y_{5k} = 0 \\ \sum_{k=1}^{3} y_{6k} = 0, \sum_{k=1}^{4} y_{7k} = 0, \sum_{k=1}^{4} y_{8k} = 0, \sum_{k=1}^{5} y_{9k} = 0, \sum_{k=1}^{6} y_{10k} = 0 \\ \sum_{i=1}^{10} \sum_{k=1}^{4} 2^{k-1} b_k^{[i]} C_i^{(1)} \le 4000 \\ \sum_{i=1}^{10} \sum_{k=1}^{4} 2^{k-1} b_k^{[i]} y_{ij} R_{ij}^* \le N_j, \forall j = 1,2,...,10 \\ n_j^{(2)} \le N_j, \forall j = 1,2,...,10 \\ R_{ij} \le R_{ij}^* \end{cases} \tag{56}$$

(2) 简化 0-1 规划模型并化为规范形

Step1：y_{ij} 取值的讨论

由于 y_{ij} 是 0-1 变量，通过对上述模型的遍历，结合 $\sum_{j=1}^{10} y_{ij} = 1, \forall i = 1,2,...,10$ 和 $\sum_{k=5}^{10} y_{1k} = 0, \sum_{k=6}^{10} y_{2k} = 0, \sum_{k=8}^{10} y_{3k} = 0, \sum_{k}^{1,9,10} y_{4k} = 0, \sum_{k}^{1,2,10} y_{5k} = 0 \sum_{k=1}^{3} y_{6k} = 0, \sum_{k=1}^{4} y_{7k} = 0, \sum_{k=1}^{4} y_{8k} = 0, \sum_{k=1}^{5} y_{9k} = 0, \sum_{k=1}^{6} y_{10k} = 0$ 等约束条件，可以得到 $\sum_{i=1}^{4} \sum_{k=1}^{4} 2^{k-1} b_k^{[i]}\ y_{ij} R_{ij}^* \le N_j, \forall j = 1,2,3$ 中的 y_{ij} 取值。最终得到：$y_{11} = y_{24} = y_{33} = y_{42} = y_{55} = y_{68} = y_{710} = y_{810} = y_{910} = y_{1010} = 1$

除上述的 10 个 y_{ij} 外，其余的 $y_{ij} = 0$。代入有关的数据。

Step2：R_{ij} 和 $n_j^{(2)}$ 取值的讨论

由于题目中要求盈利最大化，所以 $n_j^{(2)} = N_j, \forall j = 1,2,3$，$R_{ij} = R_{ij}^*$ 至此，可以写出此 0-1 规划问题的规范型（具体写出目标函数），S 为松弛变量。

$$
\begin{aligned}
\max \overline{Z} &= \overline{ers} - \sum_{k=1}^{7} \overline{op}_k \\
&= 19.2\sum_{i=1}^{10}\left(E_i V_i \sum_{k=1}^{6} 2^{k-1} b_k^{[i]} \left(\sum_{j=1}^{10} y_{ij}\right)\right) - \sum_{i=1}^{10} C_i^{(1)} \sum_{k=1}^{6} 2^{k-1} b_k^{[i]} \\
&\quad - 0.006\sum_{i=1}^{10} P_i^{(1)} \sum_{k=1}^{6} 2^{k-1} b_k^{[i]} - 0.006\sum_{i=1}^{10} M_i^{(1)} \sum_{k=1}^{6} 2^{k-1} b_k^{[i]} - 6.72\sum_{i=1}^{10} O_i^{(1)} \sum_{k=1}^{6} 2^{k-1} b_k^{[i]} \\
&\quad - 0.006\sum_{j=1}^{10} P_j^{(2)} N_j - 0.006\sum_{j=1}^{10} M_j^{(2)} N_j - 6.72\sum_{j=1}^{10} O_j^{(2)} N_j
\end{aligned}
$$

$$
s.t.\begin{cases}
b_k^{[i]} \in \{0,1\} \\
y_{11} = y_{24} = y_{33} = y_{42} = y_{55} = y_{68} = y_{710} = y_{810} = y_{910} = y_{1010} = 1; y_{ij(others)} = 0 \\
\sum_{k=1}^{6} 2^{k-1} b_k^{[i]} \in Z^+ (i = 1,2,...,10), n_j^{(2)} \in Z^+ (j = 1,2,...,10) \\
\sum_{i=1}^{10}\sum_{k=1}^{6} 2^{k-1} b_k^{[i]} C_i^{[1]} + S_1 = 2400 \\
\sum_{i=1}^{10}\sum_{k=1}^{6} 2^{k-1} b_k^{[i]} y_{ij} R_{ij}^* + S_2 = N_j, \forall j = 1,2,...,10 \\
n_j^{(2)} = N_j, \forall j = 1,2,...,10 \\
R_{ij} = R_{ij}^*
\end{cases} \tag{57}
$$

(3) 转化 0-1 规划模型的规范型为 QUBO 模型

由上述 0-1 规划模型的规范型，根据附件 1 中的有关内容将其转化为 QUBO 模型，首先将 0-1 规划模型规范型展开：

$$
\max \overline{Z} = \overline{ers} - \sum_{k=1}^{7} \overline{op}_k
$$

$$
s.t.\begin{cases}
b_k^{[i]} \in \{0,1\} \\
100\sum_{k=1}^{6} b_k^{[1]} 2^{k-1} + 140\sum_{k=1}^{6} b_k^{[2]} 2^{k-1} + 200\sum_{k=1}^{6} b_k^{[3]} 2^{k-1} + 320\sum_{k=1}^{6} b_k^{[4]} 2^{k-1} + s_{11} = 4\,000 \\
3\sum_{k=1}^{6} b_k^{[1]} 2^{k-1} R_{1j} + s_{21} = 5, 4\sum_{k=1}^{6} b_k^{[4]} 2^{k-1} R_{4j} + s_{22} = 5 \\
3\sum_{k=1}^{6} b_k^{[3]} 2^{k-1} R_{3j} + s_{23} = 5, 2\sum_{k=1}^{6} b_k^{[2]} 2^{k-1} R_{2j} + s_{24} = 5 \\
3\sum_{k=1}^{6} b_k^{[5]} 2^{k-1} R_{5j} + s_{25} = 5, 4\sum_{k=1}^{6} b_k^{[8]} 2^{k-1} R_{8j} + s_{26} = 3 \\
5\sum_{k=1}^{6} b_k^{[10]} 2^{k-1} R_{10j} + s_{27} = 3, 4\sum_{k=1}^{6} b_k^{[10]} 2^{k-1} R_{10j} + s_{28} = 3 \\
3\sum_{k=1}^{6} b_k^{[10]} 2^{k-1} R_{10j} + s_{29} = 3, 3\sum_{k=1}^{6} b_k^{[10]} 2^{k-1} R_{10j} + s_{210} = 3
\end{cases} \tag{58}
$$

松弛变量二进制转换的一般公式为：

$$
s = \sum_{k=1}^{p_0+1} 2^{k-1} b_k^{[i]} \in \{0,1,2,...,2^{p_0+1} - 1\}, b_k^{[i]} \in \{0,1\} \tag{59}
$$

给出转化后的模型目标函数如下：

由于$b_k^{[i]}$是 0-1 变量，所以$b_k^{[i]}=\left(b_k^{[i]}\right)^2$，综上所述，给出 QUBO 模型目标函数$Z$如下：

$$
\begin{aligned}
\max Z = {} & 19.2\sum_{i=1}^{10}\left(E_iV_i\sum_{k=1}^{6}2^{k-1}\left(b_k^{[i]}\right)^2\left(\sum_{j=1}^{10}y_{ij}\left(\frac{R_{ij}}{R_{ij}^*}\right)\right)\right)-\sum_{i=1}^{10}C_i^{(1)}\sum_{k=1}^{6}2^{k-1}\left(b_k^{[i]}\right)^2 \\
& -0.006\sum_{i=1}^{10}P_i^{(1)}\sum_{k=1}^{6}2^{k-1}\left(b_k^{[i]}\right)^2-0.006\sum_{i=1}^{10}M_i^{(1)}\sum_{k=1}^{6}2^{k-1}\left(b_k^{[i]}\right)^2-6.72\sum_{i=1}^{10}O_i^{(1)}\sum_{k=1}^{6}2^{k-1}\left(b_k^{[i]}\right)^2 \\
& -0.006\sum_{j=1}^{10}P_j^{(2)}N_j-0.006\sum_{j=1}^{10}M_j^{(2)}N_j-6.72\sum_{j=1}^{10}O_j^{(2)}N_j \\
& +P(100\sum_{k=1}^{6}b_k^{[1]}2^{k-1}+140\sum_{k=1}^{6}b_k^{[2]}2^{k-1}+200\sum_{k=1}^{6}b_k^{[3]}2^{k-1}+320\sum_{k=1}^{6}b_k^{[4]}2^{k-1}+s_{11}-4000)^2 \\
& +P(3\sum_{k=1}^{6}b_k^{[1]}2^{k-1}R_{1j}+s_{21}-5)^2+P(4\sum_{k=1}^{6}b_k^{[4]}2^{k-1}R_{4j}+s_{22}-5)^2 \\
& +P(3\sum_{k=1}^{6}b_k^{[3]}2^{k-1}R_{3j}+s_{23}-5)^2+P(2\sum_{k=1}^{6}b_k^{[2]}2^{k-1}R_{2j}+s_{24}-5)^2 \\
& +P(3\sum_{k=1}^{6}b_k^{[5]}2^{k-1}R_{5j}+s_{25}-5)^2+P(4\sum_{k=1}^{6}b_k^{[8]}2^{k-1}R_{8j}+s_{26}-3)^2 \\
& +P(5\sum_{k=1}^{6}b_k^{[10]}2^{k-1}R_{10j}+s_{27}-3)^2+P(4\sum_{k=1}^{6}b_k^{[10]}2^{k-1}R_{10j}+s_{28}-3)^2 \qquad (60) \\
& +P(3\sum_{k=1}^{6}b_k^{[10]}2^{k-1}R_{10j}+s_{29}-3)^2+P(3\sum_{k=1}^{6}b_k^{[10]}2^{k-1}R_{10j}+s_{210}-3)^2
\end{aligned}
$$

给定具体惩罚项P值（相对较大的数），得到具体 QUBO 模型，式(60)是一个二次型，将二次型转化为矩阵形式，得到 QUBO 模型的矩阵Q。此处Q过大（一个 70×70 的矩阵），此处的二次型过于复杂，在附件中给出，现给出形式上的 QUBO 模型表达式：

$$
\begin{aligned}
& QUBO: \max Z = b_k^{[i]}Qb_k^{[i]T} \\
& s.t. b_k^{[i]}\ \ binary
\end{aligned} \qquad (61)
$$

6.3.3 QUBO 模型的求解

对问题三的 QUBO 模型，其量子比特量超出 100，由于硬件限制，并不能直接对原 QUBO 模型进行求解，为此我们使用了基于混合模拟退火的 subQUBO 方法将原模型分解为多个子问题不断迭代求解，最终得到原模型解。

(1)基于混合模拟退火的 subQUBO 算法

• 子问题最优解等同全局最优解

将比特量为n的 QUBO 模型最优解定义为$X_{best}=\{x_1^*,x_2^*,...,x_3^*\}$。已知$X$的$n$-$m$项$X_c$，即$X_c\subseteq X_{best}$，此时$X_c$视为常数项，因此待求解变量$X$从$n$维降至$m$维度$S$,此刻模型为原模型的 subQUBO 模型。可以知 subQUBO 的最优解S一定属于X_{best},对应的 QUBO 模型表达式变化如下,(62)式为 QUBO 模型，(63)式为 subQUBO 模型。

$$f(X)=\sum_i x_i+\sum_{i\neq j}x_ix_j \qquad (62)$$

$$f_s(S)=\sum_{\substack{i\\ x_i\in S_i}}x_i+\sum_{\substack{i\neq j\\ x_i,x_j\in S}}x_ix_j+const \qquad (63)$$

其中，$x_i,C_i,const$计算如下：

$$c_i=a_i+\sum_{x_j\in X\setminus S}\hat{x}_j \qquad (64)$$

$$const = \sum a_i x + \sum b_{ij} x \tag{65}$$

(2) 基于模拟退火确定部分基态解

由(1)可知，给定QUBO模型部分基态解 X_c，即可得到subQUBO模型，求解subQUBO模型，确定未知的基态解 X_s 从而得到最终 QUBO 模型的基态解 $X_{best}(X_{best} = X_s \cap X_c)$ [2]。

确定部分基态解 X_k 的核心思想如下，给定提前需要确定的基态解个数 *n-m*,多次随机生成 *n-m* 个变量准基态解，并在初始化准基态解后对其比特量为 *m* 的 subQUBO 模型进行求解，得到该种准基太解下的局部最优解 X'_{best} 依据概率分布，在 X'_{best} 的解集中，x_{kj} 经常等于 1（或 0）则认为其基态解 $x_{kj} = 1(or0)$，为此引入变量 d_j 用来表述确定 x_j 属于基态解的概率，计算公式如下：

$$\begin{cases} c_j = \sum_{k=1}^{N_S} x_{kj} \\ d_j = \left| c_j - \frac{N_S}{2} \right| \end{cases} \tag{66}$$

由此，对 *d* 进行降序排序，选取前 *m* 个变量作为 subQUBO 模型的待求解变量，在 S_t 中随机选取 X_t 作为 subQUBO 模型中已知的基态变量，最终将其解加入准基态解集 Pool 中，重复 N_E 次最终在准基态解集中选取目标值最小的准基态解，并作为 QUBO 模型的全局最优解即基态解，算法伪代码如下:

算法 2 subQUBO **混合退火算法**

1.**procedure** METHD WITH MULTI-INSTANCES
2. **for**(i=1;i≤ N_I ;i++)do
3. $X_i \leftarrow Intialize(QUBO)$
4. $Pool \leftarrow AddIns\tan cePool(Pool, X_i)$
5. $X_{best} \leftarrow FindBest(Pool)$
6. **while** not converged **do**
7. **for** $(i = 1; i \le N_I; i++)$ **do**
8. $X_i \leftarrow Optimize(QUBO, X_i)$
9. **for** $(i = 1; i \le N_E; i++)$ **do**
10. $X_1, X_2, \ldots, X_{N_s} \leftarrow SelectIns\tan e(Pool, N_S)$
11. **for** $(j = 1; j \le n; j++)$ **do**
12. **for** $(k = 1; k \le N_s; k++)$ **do**
13. $c_j \leftarrow c_j + x_{kj}$
14. $d_j \leftarrow \left| c_j - \frac{N_s}{2} \right|$
15. $subQUBO \leftarrow Extract(ArgSort(d_1, d_2, \ldots, d_n), m, X_t)$
16. $X' \leftarrow FindBest(Pool)$
17. $Pool \leftarrow AddIns\tan cePool(Pool, X')$
18. $X_{best} \leftarrow FindBest(Pool)$
19. $Pool \leftarrow ArrangeIns\tan cePool(Pool, N_I)$
20. **return** $f(X_{best}), X_{best}$

其中，$Optimize(QUBO, X_i)$ 表示利用求解器求解一致变量 X_i 的子问题，我们依旧使用模拟退火求解器以及 CIM 模拟器对其求解，模型初始化参数与求解模型一参数相同。

最终，挖掘机与矿车的最优采购方案如下表：

表 6 挖掘机采购方案表

挖机类型	采购数量	挖机类型	采购数量
挖 1	5	挖 6	1
挖 2	2	挖 7	0
挖 3	2	挖 8	0
挖 4	1	挖 9	0
挖 5	1	挖 10	1

表 7 挖掘机与矿车最优匹配关系表

挖机类型	匹配矿车类型	挖机类型	匹配矿车类型
挖 1	矿 3	挖 6	矿 9
挖 2	矿 4	挖 7	矿 10
挖 3	矿 5	挖 8	矿 10
挖 4	矿 6	挖 9	矿 10
挖 5	矿 8	挖 10	矿 10

表 8 挖掘机与匹配矿车的具体数量表

挖机类型	匹配矿车数量	挖机类型	匹配矿车数量
挖 1	1	挖 6	2
挖 2	2	挖 7	2
挖 3	2	挖 8	0
挖 4	2	挖 9	0
挖 5	2	挖 10	3

同样对其比特演化过程及哈密顿量值进行了可视化，迭代图像如下：

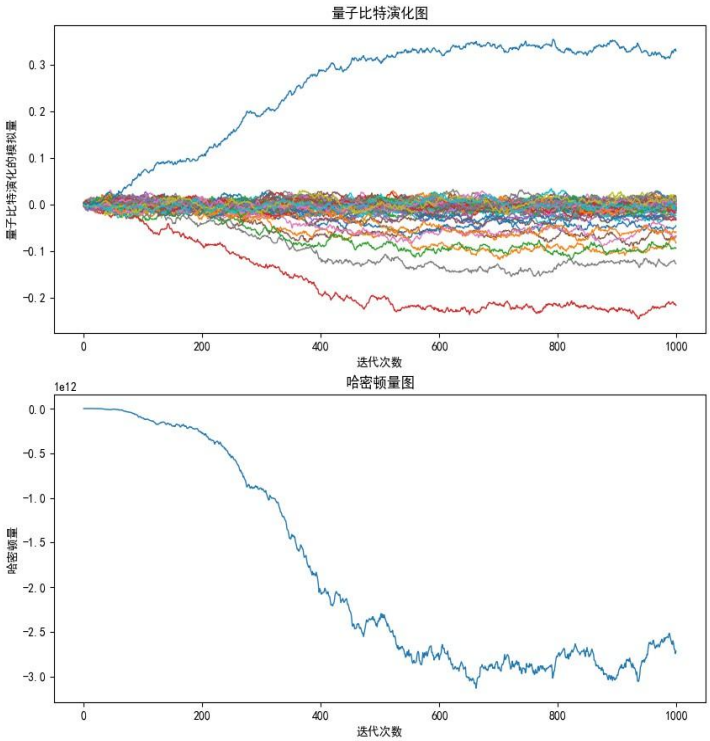

图 6 问题三 CIM 求解器的求解迭代图像

由上图可见，相较于前两问，量子比特模拟量演化过程更加复杂化，并且哈密顿量值整体上在迭代过程保持递减趋势，随着迭代过程逐渐趋于最优解，因此可见 CIM 模拟器的求解效率符合预期，其对于量子比特数较多的问题的求解具有更好效果，CIM 求解器相较于模拟退火算法更加稳定和准确。

最终我们可以得知**最优采购方案**为采购 1 型挖掘机 5 辆，2 型挖掘机 2 辆，3 型挖掘机 2 辆，4 型挖掘机 1 辆，5 型挖掘机 1 辆，6 型挖掘机 1 辆，7 型挖掘机、8 型挖掘机、9 型挖掘机不采购，10 型挖掘机 1 辆，**最大总利润为 52373.97 万元。**

其中 1 型挖掘机匹配 1 辆矿车 3，2 型挖掘机匹配 2 辆矿车 4，3 型挖掘机匹配 2 辆矿 5，4 型挖掘机匹配 2 辆矿车 6，5 型挖掘机匹配 2 辆矿车 8，6 型挖掘机匹配 2 辆矿车，10 型挖掘机匹配 3 辆矿车。

6.4 问题四的讨论——基于 QUBO 模型的实时交通信号控制优化研究

6.4.1 研究背景及意义

(1) 研究背景

习近平总书记向全球可持续交通高峰论坛致贺信指出：“建设安全、便捷、高效、绿色、经济、包容、韧性的可持续交通体系，是支撑服务经济社会高质量发展、实现‘人享其行、物畅其流’美好愿景的重要举措。建设交通强国进行合理的交通信号配时对提升交通运行效率至关重要。

随着城市化进程的推进，中国已然成为一个交通大国，截止 2023 年全国机动车保有量达 4.35 亿辆。随之而来的是愈加明显的交通压力和负担，在部分城市的上下班高峰路段以及节假日，交通拥堵、瘫痪的情况时有发生，优化交通信号控制，提升交通效率是一个亟待解决的重要问题。

在实时交通信号控制优化问题中，需要进行许多决策。在一个复杂的城市交通网中需要考虑各个路口的信号相位是否开放车辆通行，高速公路上匝道是否开放等问题。QUBO 模型可以解决此类组合优化问题，以 0-1 变量为决策变量，构建形式为二次多项式函数的目标函数。利用量子计算的方法利用模拟退火求解器和 CIM 求解器进行求解 QUBO 模型，提升具有大量 0-1 决策的优化模型的求解效率。

对城市的复杂交通网来说，量子计算以其独特的并行性和纠缠性质，为解决复杂问题提供了前所未有的机会。通过利用量子比特的量子叠加和量子纠缠，量子计算能够以极高的效率处理此类具有大规模数据交通信号配时优化问题。同时规模化的使用此类模型，为科学、工程和商业等不同领域带来了巨大的潜力和可能性。

(2) 研究意义

本研究的研究意义如下：

从经济角度看，交通拥堵不仅给城市交通运输系统带来了巨大的时间和能源损耗，还影响了城市经济的正常运行。长时间的交通拥堵导致车辆停滞不前，增加了车辆运行成本，降低了交通运输效率，进而阻碍了货物和人员的流动，影响了物流配送和商业活动。通过利用 QUBO 模型优化实时交通信号控制，可以减少交通拥堵，提高车辆通行速度，降低交通运输成本，提高运输效率，从而促进城市经济的发展。

从社会角度看，实时优化交通信号控制不仅可以提高城市居民的出行效率，减少他们的通勤时间和交通费用，还可以改善城市环境和居民生活品质。交通拥堵不仅影响了人们的生活和工作，还给城市带来了噪音、污染和安全隐患等问题。通过优化交通信号配时，可以减少交通拥堵和停车排队，缓解城市交通压力，改善交通环境，提升居民的出行体验。此外，优化交通信号配时还有利于减少交通事故发生的可能性，提高交通安全性，保障人民生命财产安全。

6.4.2 研究现状

全球范围内的交通拥堵问题日益严重，给城市交通网络带来了巨大的挑战。特别是在城市交通网络中，十字路口是交通流量聚集和分配的关键节点，其交通信号控制的效率直接影响着交通网络的整体运行效果。因此，交通控制信号是交通网络的关键组成部分，也是交通网络成功运行的主要特征。目前的研究已经引入了多种不同方式克服交通拥堵问题，包括传统的时间固定型信号控制[3]、基于需求的自适应信号控制[4]、基于交通流量的自适应信号控制[5]等。尽管这些方法在一定程度上改善了交通拥堵情况，但仍然存在一些问题，例如对交通情况变化响应不够及时、信号控制方案局限性大等。这些方法绝大多数是反应性的（当前观察到交通拥堵时才分配速度限制）。

近年来，人工智能领域的发展为交通信号控制优化带来了新的思路和方法。基于深度学习的交通信号控制方法能够利用大数据进行实时优化，但在处理复杂的交通网络时计算复杂度较高，且存在训练数据获取难、模型解释性差等问题。

构建利用大量数据的实时交通管理基础设施也是当前技术的一个重大问题，需要采取新技术来解决这些障碍。量子计算作为一种新兴的计算技术，为解决交通信号控制优化问题提供了全新的可能性。我们可以利用量子纠缠和关联这样的量子特性，使量子算法具有高效的计算加速。量子退火在组合优化问题上具有显著的优势。基于 QUBO 模型的量子优化方法尤为突出，它将交通信号控制问题转化为二次多项式目标函数的优化问题，通过量子计算的方法高效地求解。这种方法能够充分考虑交通网络的复杂性，实现实时信号控制的优化，从而提高交通网络的运行效率[6]。

虽然量子计算技术在交通信号控制优化领域的应用处于初级阶段，但已经取得了一些的成果。例如，中国台湾的 Amit Singh 团队已经通过构建 QUBO 模型[7]，利用量子优化算法对小规模交通网络进行了优化实验，取得了较好的效果，并且以台湾的“东大-科苑”交叉口进行交通流量的仿真模拟，验证了 QUBO 模型的优化效果。

本研究拟实时优化交通信号相位和交通灯的时间，以实现最大的交通流量。拟构建 QUBO 模型并使用模拟量子退火进行启发式优化。同时拟采集浙江省杭州市的交通路网及交通流数据。基于 VISSIM 搭建复杂交通路网控制系统的动态仿真模拟平台。比较分析 QUBO 模型优化前后的交通路网通行效率的提升情况，对比 QUBO 模型与其他人工智能算法的优化效果。

6.4.3 创新点及研究思路

(1)创新点

我们团队意识到交通拥堵对城市经济和居民生活的影响，因此研究交通信号优化具有**重要的实际意义**。

我们将使用大规模的交通路网数据，如浙江省杭州市的数据，确保模型和算法在实际应用中的可行性和适用性，并在结果分析中对不同规模的交通路网进行比较，展示算法在**大规模场景下的优越性**。

我们将明确使用量子优化算法（如模拟量子退火）进行求解，突出**量子计算在组合优化问题上的优势**。在结果分析中，我们将与传统算法进行对比，展示量子优化算法在求解交通信号控制优化问题上的效率和性能优势，并强调量子计算的并行性和纠缠性质，使得其在处理大规模数据交通信号配时优化问题时具有独特的优势。突显研究的重要性和实用性，以及量子计算在解决交通信号优化问题上的优势，所以该研究具有创新价值与实际意义。

(2)研究思路

• 问题定义和建模

确定研究的具体目标，例如最大化交通流量、减少交通拥堵等。目标还可能包括最大化交通流量、最小化交通延误、优化交通能耗等。在定义目标时需要考虑实际情况和

城市交通特点。建立 QUBO 模型时，需要明确定义决策变量，例如每个交叉路口的信号相位及其对应的时间分配。

• 数据采集和预处理

数据采集要确保覆盖到足够多的路段和交叉口，以反映复杂城市交通网络的真实情况。可能需要借助交通监控系统、GPS 数据等多种数据源，同时要确保数据来源的可靠性。在数据预处理时需要进行异常值、缺失值处理等工作，以确保数据的质量和可用性。

• 模型求解

在模型求解阶段，可以尝试不同的量子优化算法，比如模拟量子退火、量子变分算法等，并进行对比实验，找出最适合该问题的算法。同时在求解过程中我们需要设置合适的参数，如退火温度、退火速率等，以及控制算法迭代次数和收敛条件等。

• 仿真实验

在仿真模拟实验中，需要构建真实的交通路网模型，包括道路拓扑结构、交叉口设置、车流量等。可以利用现有的交通仿真软件 VISSIM 等进行模拟。在仿真过程中，需要设定不同的交通信号控制方案，包括传统固定时序方案、基于需求的自适应方案以及基于流量的自适应方案等，以比较不同方案的效果。

• 结果分析和比较

分析优化后的交通路网通行效率，可以通过指标如平均车速、交通延误时间、车辆停滞次数等来评价不同方案的效果。比较 QUBO 模型与其他人工智能算法的优化效果时，除了考虑优化效果外，还需要考虑算法的计算复杂度、稳定性、可扩展性等方面。

• 结论和展望

最后总结研究结果，提出对交通信号控制优化的见解和建议，探讨其对城市交通管理的实际意义和应用前景。展望未来的研究方向，可以考虑进一步优化算法、拓展研究范围、结合其他技术手段等，以提高交通信号控制的效率和可靠性。

(3) 技术路线

给出该研究的技术路线如下：

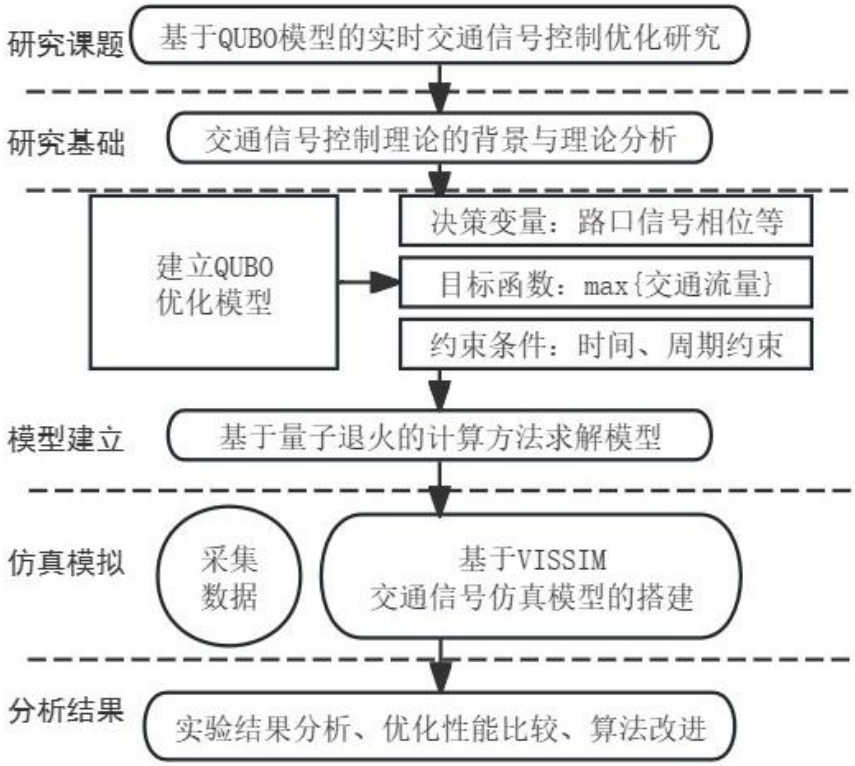

图 7 研究技术路线图

6.4.4 实时交通信号控制优化研究的 QUBO 模型

当前的 CMOS 退火机、数字退火机和量子退火机可以通过将优化问题转换为伊辛最小化问题来有效地解决优化问题，已知伊辛最小化问题在经典计算机上是 NP 困难的。$(\sigma_1,\sigma_2,...,\sigma_n)$ 表示具有双自旋的 Ising 模型的哈密顿量，h_i 表示伊辛模型的哈密尔顿算子。J_{ij} 表示二进制自旋。得到下式：

$$H(\sigma_1,\sigma_2,...,\sigma_n) = -\sum_{i<j} J_{ij}\sigma_i\sigma_j - \sum_i h_i\sigma_i \tag{67}$$

(67)式的内容实际上等同于 QUBO 模型：

$$\begin{aligned} &QUBO : O := xQx^T \\ &s.t. x \in \{0,1\} \end{aligned} \tag{68}$$

其中，x 是一个 0-1 变量，矩阵 Q 是描述它们之间相关性的矩阵。玻色量子公司开发的套件可以很容易地集成到其他伊辛机中，如数字退火机，量子退火机等。在交通信号控制中，QUBO 问题中要最小化的目标函数可以表示为：

$$O = \sum_j c_{ij}q_{ij} + \varphi\sum_i (1-\sum_j q_{ij})^2 \tag{69}$$

其中，q_{ij} 表示第 i 个交叉路口（交通灯组）处的交通灯信号模式 j（绿色、黄色、红色相位）是否开启的 0-1 变量。提供适当选择的系数 C_{ij}，交通信号模式与改善总体交通状况的潜力相关联。在这项工作中，我们选择的总停车汽车的数量 C_{ij} 在进站车道控制的信号模式 j 的第 i 个路口。通过设计的目标函数将优化系统，以清除最大数量的车辆停在十字路口前。足够大的惩罚因子 φ 确保在每个路口仅选择一个模式的约束。

给出一个普通交通十字路口的示意图：

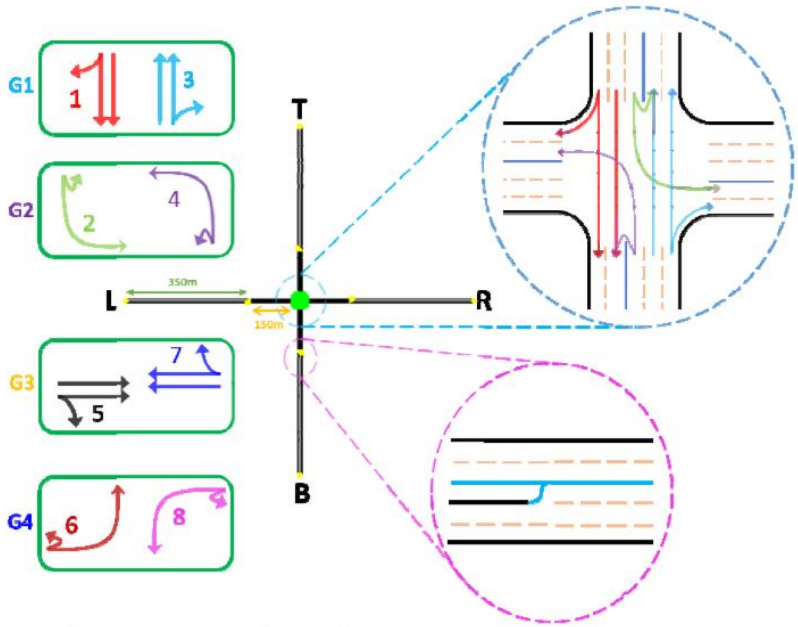

图 8 十字路口相位示意图

复杂的交通系统网路主要就是由若干个四个相位的十字路口组成，我们采取分治的思想，对上述式子路口的子模型进行优化求解。

6.4.5 预期结果与应用

本研究旨在通过最优交通信号控制方案，有效地最大化交叉口的交通流量，以缓解城市地区因交通拥堵而造成的拥堵、污染和延误。我们期望通过量子优化算法得到动态

调整交通灯时间的方案，以此为基础进一步增加城市交通网络的运行效率。通过利用 VISSIM 的模拟仿真，我们将验证模型的准确性，并确定最优交通信号控制方案在实际交通场景中的可行性和效果。

预期的应用与结果包括以下方面：首先，我们的研究成果可为城市交通管理部门提供实用的交通信号控制方案，帮助他们更有效地管理交通流量，减少拥堵和延误，提升交通运行效率。其次，通过减少交通拥堵和停车排队，我们的最优交通信号控制方案有望减少车辆排放，改善城市环境质量，降低空气污染水平。同时，优化交通信号控制将降低交通运输成本，提高运输效率，进而促进城市经济的发展。此外，最优交通信号控制方案有望降低交通事故发生的可能性，提高交通系统的安全性，有效保障人民生命财产安全。最后，我们的研究还将为在更大范围内进行交通流优化提供技术支持和理论指导，为城市甚至国家级交通管理提供决策依据和方法论支持。通过本研究的成果，我们将为实现城市交通系统的智能化、高效化提供重要的科学支撑和技术保障，为城市可持续发展和居民生活品质的提升做出贡献。

七、量子计算求解效率分析

在求解 QUBO 模型时，使用模拟退火求解器和 CIM 模拟器两种方法对模型求解，发现 CIM 性能上与模拟退火算法有一定差异，为进一步研究算法之间求解效率的差异，基于前文建立的 3 个 QUBO 模型，将其作为小、中、大三种规模的实例，以此研究量子计算与模拟退火在不同规模性的性能差异，最终发现量子计算的优势如下：

7.1.较短的求解时间

为比较量子计算与模拟退火对不同规模问题运行时间上的差异，我们分别用 CIM 模拟器和模拟退火求解器对不同规模问题重复求解 100 次，并计算每种规模问题下的运行时间的平均值，以此比较两者的求解速率，最终得到如下图：

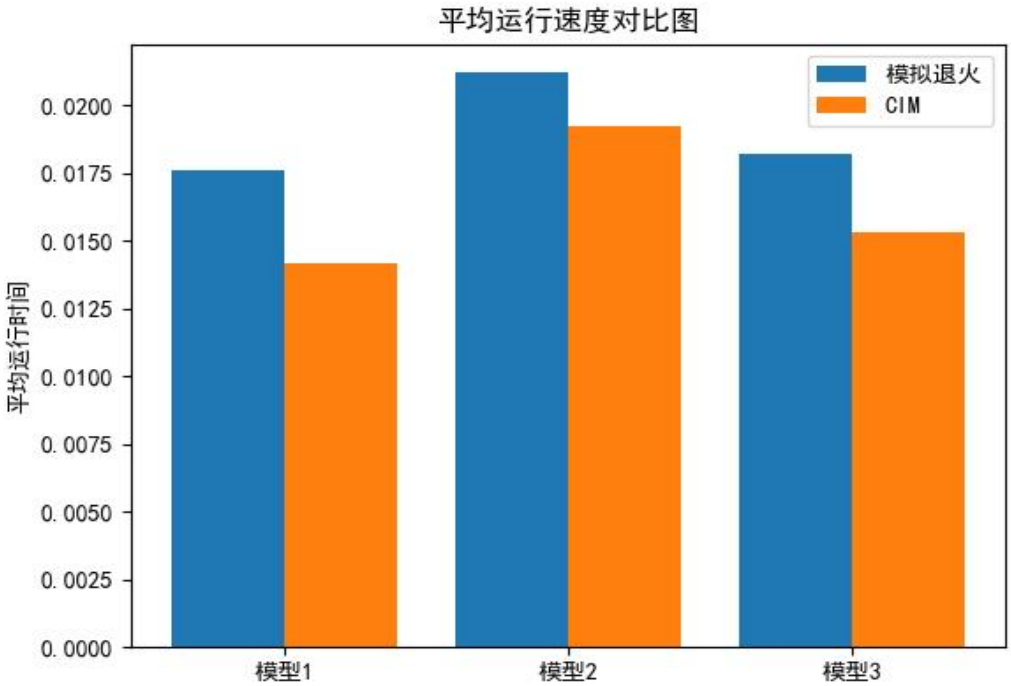

图 9 求解各规模问题求解速率对比图

由图 9 可以看出，CIM 模拟器对于小、中、大三种规模的求解速度都优于模拟退火求解器，考虑到我们使用的只是 CIM 的模拟器，如若使用真实的 CIM 器效果会更优。

7.2 较稳定的最优解

比较量子计算与模拟退火单次求解模型得到解的准确性，依旧使用 CIM 模拟器以及模拟退火求解器对不同规模问题重复求解 100 次，并记录解的最优值最终绘制图像如下：

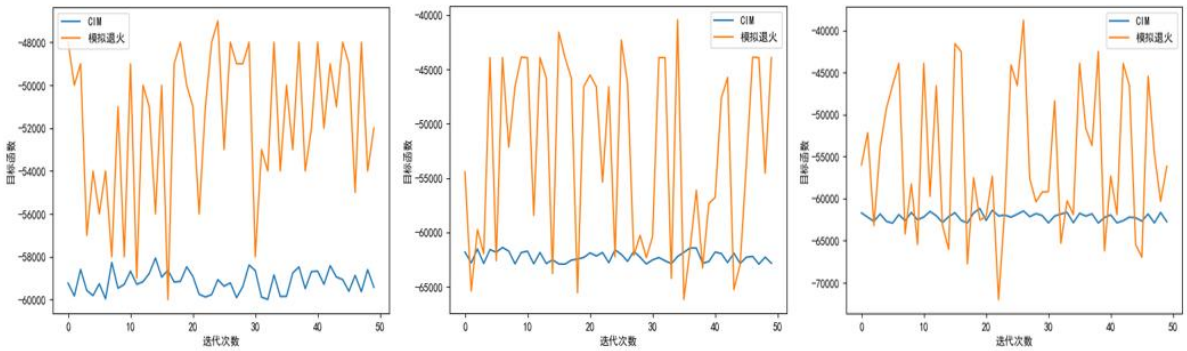

图 10 模型求解波动图

图 10 中，左至右依次为小、中、大三个规模的求解波动图。由上图可以看出，量子计算求解各种规模问题的**准确性和稳定性**均明显优于模拟退火算法，其每次解与最优解偏差较小；反观模拟退火算法波动较大，需要确定最优解的次数过多。通过比较两种方法求解的速率以及准确性，我们可以看出量子计算在求解复杂优化问题的效果明显优于传统的模拟退火算法，并且使用物理 CIM 硬件求解的效果将更加符合预期。

八、模型评价与改进

8.1 模型的优点

1.本题我们建立了以最大化总利润为目标的优化模型，并基于二进制编码的思想转化成为 QUBO 模型，对于问题一和问题二，该模型的量子比特数较少，使用模拟退火求解器和 CIM 模拟器所得结果相近，并且收敛速度较快，避免了局部最优解的情况。

2.本题建立的规划模型，对题目所给约束全面地进行了数学模型的转化，并且目标函数包含各个约束的限制，对于该问题具有很好概括性，同时我们尽可能减少 0-1 变量的出现，对于模型的求解速度具有不错的提升。

3.在问题三中，由于模型的量子比特数较大，我们采用基于混合退火的算法将模型的规模分解成各个子问题进行求解，对于模型的求解效率有不错的提升，并且避免了大规模问题在求解器中不可解的情况，是一种不错的创新求解思路。

8.2 模型的缺点

1.在决策变量增多的情形下，问题三的优化模型转成 QUBO 模型时具有较大的量子比特数，这对于一般的求解器会产生不可解的弊端，因此优化模型的量子比特数是解决该问题的最直接的手段，而本文并没有找到更加合适的模型简化方案。

2.对于问题三的挖掘机购买方案，虽然符合约束条件，但可能并非最优解，由于求解器的局限性，在 CIM 模拟求解过程中可能会产生优于该解的答案，因此我们不能保证问题三答案的完全正确性。

8.3 模型的改进

在将以最大化总利润为目标的 0-1 规划模型转化成 QUBO 模型的过程中，我们引入了松弛变量来建立等式约束，对于松弛变量的取值范围，我们在模型中并未考虑，而是直接将其用给定足够大数目的二进制变量来表示，这对于模型的求解存在弊端，会增加 QUBO 模型的量子比特量，造成求解效率变低。

为了避免量子比特数过多的情况，我们可以根据模型所给的约束条件对松弛变量的取值范围进行进一步的估计，这可一定程度的减少量子比特量，优化模型的求解效率以及结果的准确性。

九、模型的推广与应用

本文建立了解决矿山设备配置及运营方案优化问题的数学模型，该模型可以应用于研究其他与配比及运营相关的优化问题，由于本题对于大规模变量的求解使用了量子计算求解，不仅使得求解效率大幅提升，对于现实中较多大规模的优化问题也具有很好的推广作用，也为许多 NP-Hard 问题的求解提供了参考方式，具有重要的现实意义和应用价值。

参考文献

[1] Fred Glover, Gary Kochenberger, Yu Du “Quantum Bridge Analytics I: A Tutorial on Formulating and Using QUBO Models”

[2] Yuta Atobe , Masashi Tawada,Nozomu Togawa “Hybrid Annealing Method Based on subQUBO Model Extraction With Multiple Solution Instances”

[3] D. Venturelli, D. J. J. Marchand, and G. Rojo, “Quantum Annealing Implementation of Job-Shop Scheduling,” pp. 1–15, 2015.

[4] H. Hussain, M. Bin Javaid, F. S. Khan, A. Dalal, and A. Khalique, "Optimal control of traffic signals using quantum annealing," Quantum Inf. Process., vol. 19, no. 9, p. 312, 2020.

[5] S. Yarkoni et al., "Quantum Shuttle: Traffic navigation with Quantum computing," APEQS 2020 - Proc. 1st ACM SIGSOFT Int. Work. Archit. Paradig. Eng. Quantum Software, Co-located with ESEC/FSE 2020, pp. 22–30, 2020.

[6] F. Neukart, G. Compostella, C. Seidel, D. von Dollen, S. Yarkoni, and B. Parney, "Traffic Flow Optimization Using a Quantum Annealer," Front. ICT, vol. 4, p. 29, 2017.

[7] Amit Singh, Chun-Yu Lin, Chung-I Huang, and Fang-Pang Lin, “Quantum Annealing Approach for the Optimal Real-time Traffic Control using QUBO,” Eur. Transp. Res. Rev., vol. 12, no. 1, p. 55, 2020.

附录

问题一的参考代码：pro_1_CIM.py，pro_1_SA

介绍： CIM 模拟器代码，模拟退火求解器代码

```
''' CIM 求解器 '''
import kaiwu as kw
import numpy as np
import matplotlib.pyplot as plt
plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False        # 显示负号

# 授权初始化代码
# 示例的 user_id 和 sdk_code 无效，需要替换成自己的用户 ID 和 SDK 授权码
kw.license.init(user_id="39882159518912514",
sdk_code="LjDkyaZpeF9IK6va2xPLsQ7KdxuD59")
# 需要 16 个二进制变量
n = 4
# 创建 qubo 模型的 0-1 变量矩阵
x = kw.qubo.ndarray((n, n), "x", kw.qubo.Binary)
# 给定所有约束条件
cons1 = (100*kw.qubo.quicksum(x[0, k]*pow(2, k) for k in range(0, n))
       + 140*kw.qubo.quicksum(x[1, k]*pow(2, k) for k in range(0, n))
       + 200*kw.qubo.quicksum(x[2, k]*pow(2, k) for k in range(0, n))
       + 320*kw.qubo.quicksum(x[3, k]*pow(2, k) for k in range(0, n)) - 2400)**2
cons2 = (kw.qubo.quicksum(x[0, k]*pow(2, k) for k in range(0, n)) - 7)**2
cons3 = (kw.qubo.quicksum(x[1, k]*pow(2, k) for k in range(0, n)) - 7)**2
cons4 = (kw.qubo.quicksum(x[2, k]*pow(2, k) for k in range(0, n)) +
kw.qubo.quicksum(x[3, k]*pow(2, k) for k in range(0, n)) - 3)**2
# 写出目标函数（转化最小目标）
min_main = (2000*kw.qubo.quicksum(pow(x[0, k],2)*pow(2, k) for k in range(0,
n))
          + 3000*kw.qubo.quicksum(pow(x[1, k],2)*pow(2, k) for k in range(0, n))
          + 5000*kw.qubo.quicksum(pow(x[2, k],2)*pow(2, k) for k in range(0, n))
          + 6000*kw.qubo.quicksum(pow(x[3, k],2)*pow(2, k) for k in range(0,
n)))*(-1)
# 写出 qubo 形式的数学模型/最终目标函数（加入惩罚项）
qubo_main = min_main + 100000*(cons1 + cons2 + cons3 + cons4)
# 解析 qubo
qubo_main = kw.qubo.make(qubo_main)
# 转换为 ising 模型
qubo_main_ising = kw.qubo.cim_ising_model(qubo_main)
# 提取伊辛矩阵
matrix = qubo_main_ising.get_ising()["ising"]
# 提取 qubo 矩阵
qubo_matrix = kw.qubo.qubo_model_to_qubo_matrix(qubo_main)
array = qubo_matrix["qubo_matrix"]
```

```
print(array.shape)
# CIM求解器
output = kw.cim.simulator(
                matrix,
                pump = 1.3,
                noise = 0.2,
                laps = 5000,
                dt = 0.05,
                normalization = 0.3,
                iterations = 50)
'''
# 模拟退火求解器
worker = kw.classical.SimulatedAnnealingOptimizer(
                                    initial_temperature=100,
                                    alpha=0.99,
                                    cutoff_temperature=0.001,
                                    iterations_per_t=10,
                                    size_limit=50)
# ising模型的结果矩阵(+1或-1)
output = worker.solve(matrix)
'''
# 对结果排序
opt = kw.sampler.optimal_sampler(matrix, output, bias=0, negtail_ff=False)
# 查看量子比特演化
matrix_n = kw.cim.normalizer(matrix, normalization=0.5)
output_bite = kw.cim.simulator_core(
            matrix_n,
            c = 0,
            pump = 0.7,
            noise = 0.01,
            laps = 1000,
            dt = 0.1)
# 计算哈密顿量,一般使用未进行归一化的矩阵进行计算
h = kw.sampler.hamiltonian(matrix, output_bite)
plt.figure(figsize=(10, 10))

# 脉冲图
plt.subplot(211)
plt.plot(output_bite, linewidth=1)
plt.title("量子比特演化图")
plt.ylabel("量子比特演化的模拟量")
plt.xlabel("迭代过程")

# Energy diagram
plt.subplot(212)
plt.plot(h, linewidth=1)
```

```
plt.title("哈密顿量图")
plt.ylabel("哈密顿量")
plt.xlabel("迭代过程")
plt.savefig('./bite_fig_pro1.png')
plt.show()
# 选择最佳解决方案
cim_best = opt[0][0]
# 如果线性项变量为-1，则执行翻转
cim_best = cim_best * cim_best[-1]

print(cim_best)

# 我们使用 get_val 模块进行还原 spin 结果为变量值
# ising 矩阵变量赋索引值
vars = qubo_main_ising.get_variables()

# 替换自旋向量并获得结果字典(ising 转 qubo 的解向量)
sol_dict = kw.qubo.get_sol_dict(cim_best, vars)

# 检查硬约束的有效性和路径长度
cons1_val = kw.qubo.get_val(cons1, sol_dict)
cons2_val = kw.qubo.get_val(cons2, sol_dict)
cons3_val = kw.qubo.get_val(cons3, sol_dict)
cons4_val = kw.qubo.get_val(cons4, sol_dict)

print('cons1: {}'.format(cons1_val))
print('cons2: {}'.format(cons2_val))
print('cons3: {}'.format(cons3_val))
print('cons4: {}'.format(cons4_val))

# 使用 min_main 计算最大化总利润目标函数值
min_val = kw.qubo.get_val(min_main, sol_dict)
print('min_val: {}'.format(min_val))
```

问题二的参考代码：pro_2_CIM.py，pro_2_SA

介绍： CIM 模拟器代码，模拟退火求解器代码

```
import kaiwu as kw
import numpy as np
import matplotlib.pyplot as plt
plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False        # 显示负号
# 授权初始化代码
# 示例的 user_id 和 sdk_code 无效，需要替换成自己的用户 ID 和 SDK 授权码
kw.license.init(user_id="39882159518912514",
sdk_code="LjDkyaZpeF9IK6va2xPLsQ7KdxuD59")
# 挖掘机参数
```

```
E = np.array([190, 175, 165, 150])
V = np.array([0.9, 1.2, 1.8, 2.1])
O1 = np.array([28, 30, 34, 38])
C1 = np.array([100, 140, 200, 320])
P1 = np.array([0.7, 0.75, 0.85, 0.9])
M1 = np.array([0.1, 0.15, 0.2, 0.3])
# 矿车参数
O2 = np.array([18, 22, 27])
P2 = np.array([0.6, 0.7, 0.8])
M2 = np.array([0.2, 0.3, 0.4])
N = np.array([7, 7, 3])
# 需要 16 个二进制变量
n = 4
m = 3
# 创建 qubo 模型的 0-1 变量矩阵
x = kw.qubo.ndarray((n, n), "x", kw.qubo.Binary)
y = kw.qubo.ndarray((4, 3), "x", kw.qubo.Binary)
# 惩罚项约束
cons1 = (100*kw.qubo.quicksum(x[0, k]*pow(2, k) for k in range(0, n))
      + 140*kw.qubo.quicksum(x[1, k]*pow(2, k) for k in range(0, n))
      + 200*kw.qubo.quicksum(x[2, k]*pow(2, k) for k in range(0, n))
      + 320*kw.qubo.quicksum(x[3, k]*pow(2, k) for k in range(0, n)) - 2400)**2
cons2 = (kw.qubo.quicksum(x[0, k]*pow(2, k) for k in range(0, n)) - 7)**2
cons3 = (kw.qubo.quicksum(x[1, k]*pow(2, k) for k in range(0, n)) - 7)**2
cons4 = (kw.qubo.quicksum(x[2, k]*pow(2, k) for k in range(0, n)) - 2)**2
cons5 = (kw.qubo.quicksum(x[3, k]*pow(2, k) for k in range(0, n)) - 1)**2
# 写出目标函数（转化最小目标）
min_main  =  (19.2*kw.qubo.quicksum(E[i]*V[i]*(kw.qubo.quicksum(pow(x[i,  k],
2)*pow(2, k) for k in range(n))) for i in range(n))
           - kw.qubo.quicksum(C1[i]*kw.qubo.quicksum(pow(x[i,  2], 2)*pow(2,
k) for k in range(n)) for i in range(n))
           -    0.006*kw.qubo.quicksum(P1[i]*kw.qubo.quicksum(pow(x[i,    k],
2)*pow(2, k) for k in range(n)) for i in range(n))
           -    0.006*kw.qubo.quicksum(M1[i]*kw.qubo.quicksum(pow(x[i,    k],
2)*pow(2, k) for k in range(n)) for i in range(n))
           -     6.72*kw.qubo.quicksum(O1[i]*kw.qubo.quicksum(pow(x[i,    k],
2)*pow(2, k) for k in range(n)) for i in range(n))
           - 0.006*kw.qubo.quicksum(P2[j]*N[j] for j in range(m))
           - 0.006*kw.qubo.quicksum(M2[j]*N[j] for j in range(m))
           - 6.72*kw.qubo.quicksum(O2[j]*N[j] for j in range(m)))*(-1)

# 写出 qubo 形式的数学模型/最终目标函数（加入惩罚项）
qubo_main = min_main + 100000*(cons1 + cons2 + cons3 + cons4 + cons5)

# 解析 qubo
qubo_main = kw.qubo.make(qubo_main)
```

```
# 转换为ising模型
qubo_main_ising = kw.qubo.cim_ising_model(qubo_main)
# 提取伊辛矩阵
matrix = qubo_main_ising.get_ising()["ising"]
# 提取qubo矩阵
qubo_matrix = kw.qubo.qubo_model_to_qubo_matrix(qubo_main)
array = qubo_matrix["qubo_matrix"]
print(array.shape)
# CIM求解器
output = kw.cim.simulator(
                matrix,
                pump = 1.3,
                noise = 0.2,
                laps = 5000,
                dt = 0.05,
                normalization = 0.3,
                iterations = 50)
'''
# 模拟退火求解器
worker = kw.classical.SimulatedAnnealingOptimizer(
                                    initial_temperature=100,
                                    alpha=0.99,
                                    cutoff_temperature=0.001,
                                    iterations_per_t=10,
                                    size_limit=50)
# ising模型的结果矩阵(+1或-1)
output = worker.solve(matrix)
'''
# 对结果排序
opt = kw.sampler.optimal_sampler(matrix, output, bias=0, negtail_ff=False)
# 查看量子比特演化
matrix_n = kw.cim.normalizer(matrix, normalization=0.5)
output_bite = kw.cim.simulator_core(
            matrix_n,
            c = 0,
            pump = 0.7,
            noise = 0.01,
            laps = 1000,
            dt = 0.1)
# 计算哈密顿量,一般使用未进行归一化的矩阵进行计算
h = kw.sampler.hamiltonian(matrix, output_bite)
plt.figure(figsize=(10, 10))

# 脉冲图
plt.subplot(211)
plt.plot(output_bite, linewidth=1)
```

```
plt.title("量子比特演化图")
plt.ylabel("量子比特演化的模拟量")
plt.xlabel("迭代过程")

# Energy diagram
plt.subplot(212)
plt.plot(h, linewidth=1)
plt.title("哈密顿量图")
plt.ylabel("哈密顿量")
plt.xlabel("迭代过程")
plt.savefig('./bite_fig_pro2.png')
plt.show()
# 选择最佳解决方案
cim_best = opt[0][0]
# 如果线性项变量为-1，则执行翻转
cim_best = cim_best * cim_best[-1]

print(cim_best)

# 我们使用 get_val 模块进行还原 spin 结果为变量值
# ising 矩阵变量赋索引值
vars = qubo_main_ising.get_variables()

# 替换自旋向量并获得结果字典(ising 转 qubo 的解向量)
sol_dict = kw.qubo.get_sol_dict(cim_best, vars)

# 检查硬约束的有效性和路径长度
cons1_val = kw.qubo.get_val(cons1, sol_dict)
cons2_val = kw.qubo.get_val(cons2, sol_dict)
cons3_val = kw.qubo.get_val(cons3, sol_dict)
cons4_val = kw.qubo.get_val(cons4, sol_dict)

print('cons1: {}'.format(cons1_val))
print('cons2: {}'.format(cons2_val))
print('cons3: {}'.format(cons3_val))
print('cons4: {}'.format(cons4_val))

# 使用 min_main 计算最大化总利润目标函数值
min_val = kw.qubo.get_val(min_main, sol_dict)
print('min_val: {}'.format(min_val))
```

问题三的参考代码：pro_3_CIM.py，pro_3_SA

介绍： CIM 模拟器代码，模拟退火求解器代码

```
import kaiwu as kw
import numpy as np
```

```
import matplotlib.pyplot as plt
plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False        # 显示负号
# 授权初始化代码
# 示例的user_id和sdk_code无效，需要替换成自己的用户ID和SDK授权码
kw.license.init(user_id="39882159518912514",
sdk_code="LjDkyaZpeF9IK6va2xPLsQ7KdxuD59")
# 挖掘机参数
E = np.array([190, 175, 165, 150, 140, 130, 120, 110, 105, 100])
V = np.array([0.9, 1.2, 1.8, 2.1, 2.6, 3.5, 5, 6, 8, 10])
O1 = np.array([28, 30, 34, 38, 42, 50, 60, 75, 90, 100])
C1 = np.array([100, 140, 200, 320, 440, 500, 640, 760, 860, 1000])
P1 = np.array([0.7, 0.75, 0.85, 0.9, 1, 1.2, 1.3, 1.6, 1.8, 2])
M1 = np.array([0.1, 0.15, 0.2, 0.3, 0.5, 0.8, 1, 1.3, 1.5, 1.8])
# 矿车参数
O2 = np.array([15, 18, 22, 27, 33, 40, 50, 55, 64, 70])
P2 = np.array([0.5, 0.6, 0.7, 0.8, 0.9, 1, 1.1, 1.2, 1.3, 1.5])
M2 = np.array([0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1])
N = np.array([5, 5, 5, 5, 5, 3, 3, 3, 3, 3])
# 需要60个二进制变量
# 创建qubo模型的0-1变量矩阵
x = kw.qubo.ndarray((10, 6), "x", kw.qubo.Binary)
cons1 = (100*kw.qubo.quicksum(x[0, k]*pow(2, k) for k in range(6))
      + 140*kw.qubo.quicksum(x[1, k]*pow(2, k) for k in range(6))
      + 200*kw.qubo.quicksum(x[2, k]*pow(2, k) for k in range(6))
      + 320*kw.qubo.quicksum(x[3, k]*pow(2, k) for k in range(6)) - 4000)**2
cons2 = (3*kw.qubo.quicksum(x[0, k]*pow(2, k) for k in range(6))-5)**2
cons3 = (2*kw.qubo.quicksum(x[1, k]*pow(2, k)*2 for k in range(6))-5)**2
cons4 = (3*kw.qubo.quicksum(x[2, k]*pow(2, k)*2 for k in range(6))-5)**2
cons5 = (4*kw.qubo.quicksum(x[3, k]*pow(2, k)*2 for k in range(6))-5)**2
cons6 = (3*kw.qubo.quicksum(x[4, k]*pow(2, k)*2 for k in range(6))-5)**2
cons7 = (5*kw.qubo.quicksum(x[5, k]*pow(2, k)*2 for k in range(6))-3)**2
cons8 = (3*kw.qubo.quicksum(x[6, k]*pow(2, k)*2 for k in range(6))-3)**2
cons9 = (3*kw.qubo.quicksum(x[7, k]*pow(2, k)*2 for k in range(6))-3)**2
cons10 = (3*kw.qubo.quicksum(x[8, k]*pow(2, k)*2 for k in range(6))-3)**2
cons11 = (3*kw.qubo.quicksum(x[9, k]*pow(2, k)*2 for k in range(6))-3)**2
# 写出目标函数（转化最小目标）
min_main  =  (19.2*kw.qubo.quicksum(E[i]*V[i]*(kw.qubo.quicksum(pow(x[i,  k],
2)*pow(2, k) for k in range(6))) for i in range(10))
           - kw.qubo.quicksum(C1[i]*kw.qubo.quicksum(pow(x[i,  2],  2)*pow(2,
k) for k in range(6)) for i in range(10))
           -    0.006*kw.qubo.quicksum(P1[i]*kw.qubo.quicksum(pow(x[i,    k],
2)*pow(2, k) for k in range(6)) for i in range(10))
           -    0.006*kw.qubo.quicksum(M1[i]*kw.qubo.quicksum(pow(x[i,    k],
2)*pow(2, k) for k in range(6)) for i in range(10))
           -     6.72*kw.qubo.quicksum(O1[i]*kw.qubo.quicksum(pow(x[i,    k],
```

```
2)*pow(2, k) for k in range(6)) for i in range(10))
            - 0.006*kw.qubo.quicksum(P2[j]*N[j] for j in range(10))
            - 0.006*kw.qubo.quicksum(M2[j]*N[j] for j in range(10))
            - 6.72*kw.qubo.quicksum(O2[j]*N[j] for j in range(10)))*(-1)
# 写出 qubo 形式的数学模型/最终目标函数（加入惩罚项）
qubo_main = min_main + 100000*(cons1 + cons2 + cons3 + cons4 + cons5 +
                               cons6 + cons7 + cons8 + cons9 + cons10 + cons11)

# 解析 qubo
qubo_main = kw.qubo.make(qubo_main)
# 转换为 ising 模型
qubo_main_ising = kw.qubo.cim_ising_model(qubo_main)
# 提取伊辛矩阵
matrix = qubo_main_ising.get_ising()["ising"]
# 提取 qubo 矩阵
qubo_matrix = kw.qubo.qubo_model_to_qubo_matrix(qubo_main)
array = qubo_matrix["qubo_matrix"]
print(array.shape)
# CIM 求解器
output = kw.cim.simulator(
                matrix,
                pump = 1.3,
                noise = 0.2,
                laps = 5000,
                dt = 0.05,
                normalization = 0.3,
                iterations = 50)
'''
# 模拟退火求解器
worker = kw.classical.SimulatedAnnealingOptimizer(
                                    initial_temperature=100,
                                    alpha=0.99,
                                    cutoff_temperature=0.001,
                                    iterations_per_t=10,
                                    size_limit=50)
# ising 模型的结果矩阵(+1 或-1)
output = worker.solve(matrix)
'''
# 对结果排序
opt = kw.sampler.optimal_sampler(matrix, output, bias=0, negtail_ff=False)
# 查看量子比特演化
matrix_n = kw.cim.normalizer(matrix, normalization=0.5)
output_bite = kw.cim.simulator_core(
            matrix_n,
            c = 0,
            pump = 0.7,
```

```
            noise = 0.01,
            laps = 1000,
            dt = 0.1)
# 计算哈密顿量,一般使用未进行归一化的矩阵进行计算
h = kw.sampler.hamiltonian(matrix, output_bite)
plt.figure(figsize=(10, 10))

# 脉冲图
plt.subplot(211)
plt.plot(output_bite, linewidth=1)
plt.title("量子比特演化图")
plt.ylabel("量子比特演化的模拟量")
plt.xlabel("迭代过程")

# Energy diagram
plt.subplot(212)
plt.plot(h, linewidth=1)
plt.title("哈密顿量图")
plt.ylabel("哈密顿量")
plt.xlabel("迭代过程")
plt.savefig('./bite_fig_pro3.png')
plt.show()
# 选择最佳解决方案
cim_best = opt[0][0]
# 如果线性项变量为-1，则执行翻转
cim_best = cim_best * cim_best[-1]

print(cim_best)

# 我们使用 get_val 模块进行还原 spin 结果为变量值
# ising 矩阵变量赋索引值
vars = qubo_main_ising.get_variables()

# 替换自旋向量并获得结果字典(ising 转 qubo 的解向量)
sol_dict = kw.qubo.get_sol_dict(cim_best, vars)

# 检查硬约束的有效性和路径长度
cons1_val = kw.qubo.get_val(cons1, sol_dict)
cons2_val = kw.qubo.get_val(cons2, sol_dict)
cons3_val = kw.qubo.get_val(cons3, sol_dict)
cons4_val = kw.qubo.get_val(cons4, sol_dict)

print('cons1: {}'.format(cons1_val))
print('cons2: {}'.format(cons2_val))
print('cons3: {}'.format(cons3_val))
print('cons4: {}'.format(cons4_val))
```

```
# 使用 min_main 计算最大化总利润目标函数值
min_val = kw.qubo.get_val(min_main, sol_dict)
print('min_val: {}'.format(min_val))
```